Recap

dimensionality reduction.

- PCA used for?

$X = [\quad]$ $\qquad$ $Y = [X]$ target

- PCA supervised or unsupervised?

x space d $\longrightarrow$ z space d

Supervised dimensionality reduction - Forward feature Selection

Best 2 features
Reduce dimensions
Conserving 90% of original accuracy

0.9×0.92

•Start with **no features**.
•Add features **one at a time** — at each step, choose the feature that gives the biggest improvement to model performance (e.g., accuracy, R², AIC).
•Stop when adding new features no longer improves performance beyond threshold.

$X =$ ht. wt. e. f. $[\quad]$ $\rightarrow$ model $\rightarrow$ 92% accuracy

ht $[\quad]$ $\rightarrow$ NB $\rightarrow$ 82%

wt $[\quad]$ 70%

e $[\quad]$ 50%

f $[\quad]$ 35%

Georgia Tech

h w e f

hwe hef hwf

Backward Feature Selection

- Start with **all features**.
- Remove features **one at a time** — at each step, drop the feature whose removal least reduces model performance.
- Stop when removing any more features harms performance too much.

Both forward and backward feature selection are quite computationally expensive

more computationally expensive

Georgia Tech

Machine Learning CS 4641-7641

Linear Regression

Nimisha Roy
Georgia Tech

These slides are inspired based on slides from Mahdi Roozbahani, Le Song, Chao Zhang, Yaser Abu-Mostafa, Andrew Zisserman.

Outline

- Supervised Learning
- Linear Regression
- Extension

$X_{n \times d} =$

Y (target)

$n \times 1$

$n =$ number of data points

$d =$ number of dimensions/features

Supervised Learning: Overview

Functions $\mathcal{F}$

$f : \mathcal{X} \rightarrow \mathcal{Y}$

Training data

$\{(x^{\{i\}}, y^{\{i\}}) \in \mathcal{X} \times \mathcal{Y}\}$

model $f(\theta)$

LEARNING

find $\hat{f} \in \mathcal{F}$

s.t. $y_a \approx \hat{f}(x^{\{i\}})$

actual target

Learning machine

New data

x

PREDICTION

$\hat{y}_p = \hat{f}(x^{\{i\}})$

y predicted

Supervised Learning: Split of data

Training Set

70% of dataset

- **Purpose:** Used to **fit** the model — the algorithm learns patterns, relationships, and weights from this data.
- **What happens here:** The model adjusts its internal parameters to minimize error.
- **Analogy:** Like studying the material before an exam.

Validation Set

10%

- **Purpose:** Used to **tune hyperparameters** and make design decisions (e.g., how many layers, what learning rate, etc.).
- **What happens here:** The model is not trained on this data; instead, it helps to evaluate how changes affect performance, preventing overfitting.
- **Analogy:** Like taking practice tests to decide which study strategy works best.

Test Set

20%

- **Purpose:** Used to **assess the final performance** of the model after all training and tuning.
- **What happens here:** No further learning or tweaking — it provides an **unbiased estimate** of the model's generalization ability.
- **Analogy:** Like the final exam — measures how well you really learned the material.

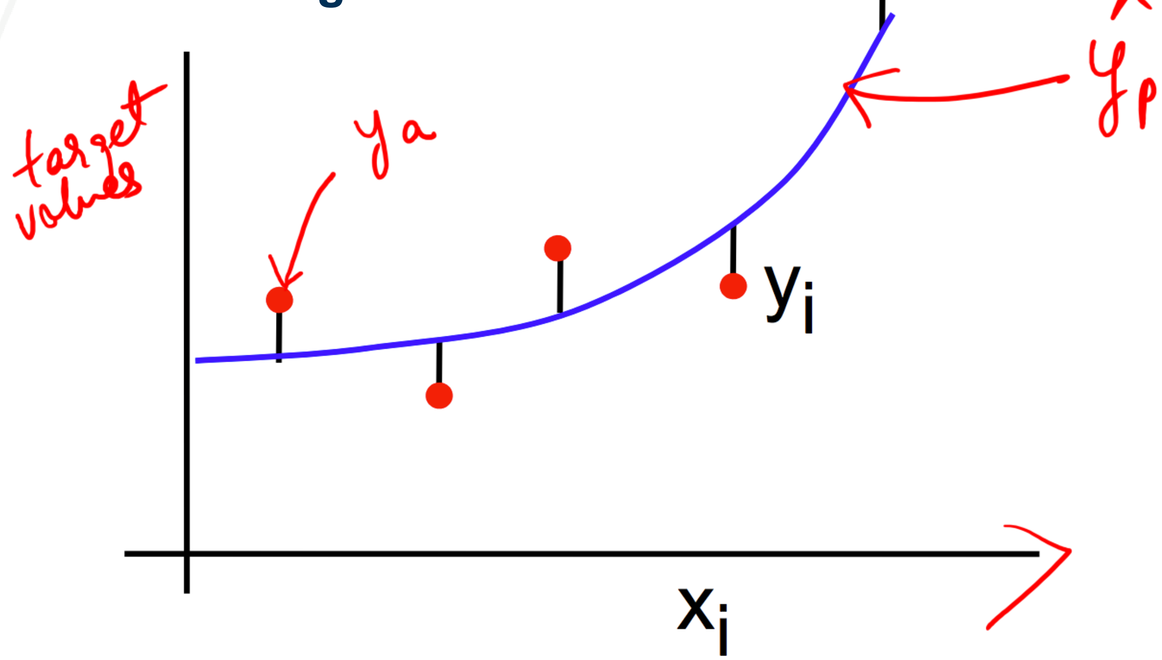
Supervised Learning: Two Types of Tasks

Given: training data $\{(x^{\{1\}}, y^{\{1\}}), (x^{\{2\}}, y^{\{2\}}), \ldots, (x^{\{n\}}, y^{\{n\}})\}$

Learn: a function $f(\mathbf{x}) : y = f(\mathbf{x})$

When y is continuous:

1. Regression

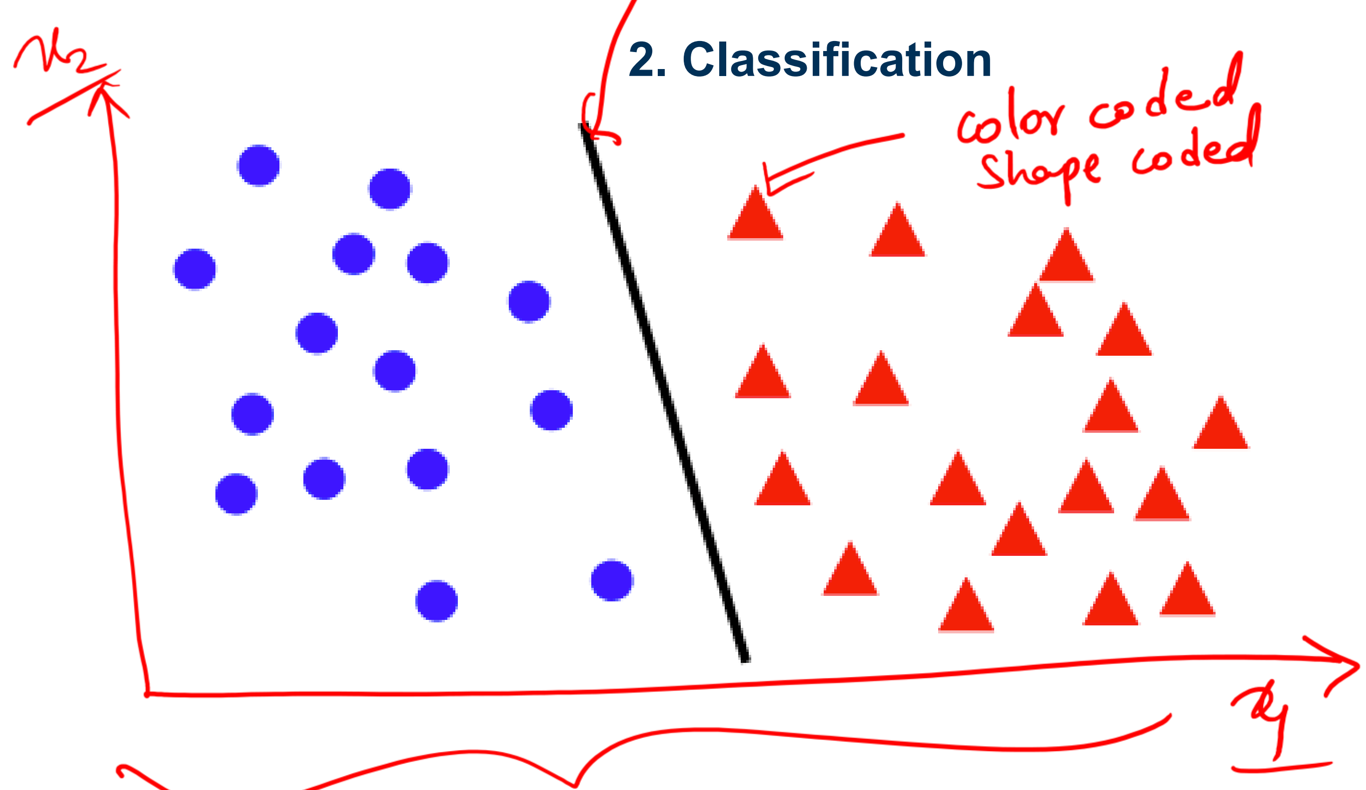


Curve fitting

decision line

When y is discrete:

2. Classification



2.5 D

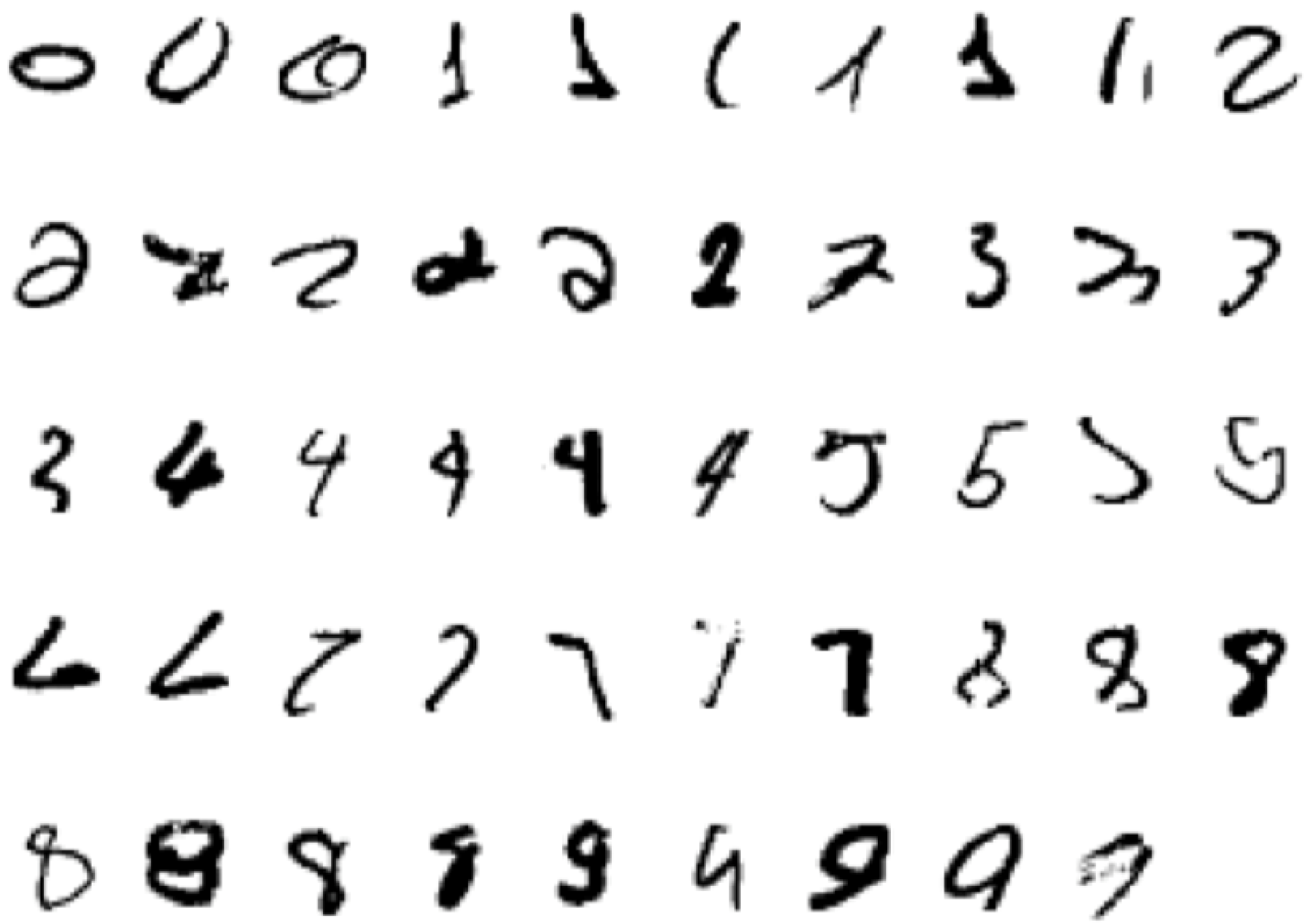
Class estimation

Classification Example 1: Handwritten digit recognition

As a supervised classification problem

10 classes

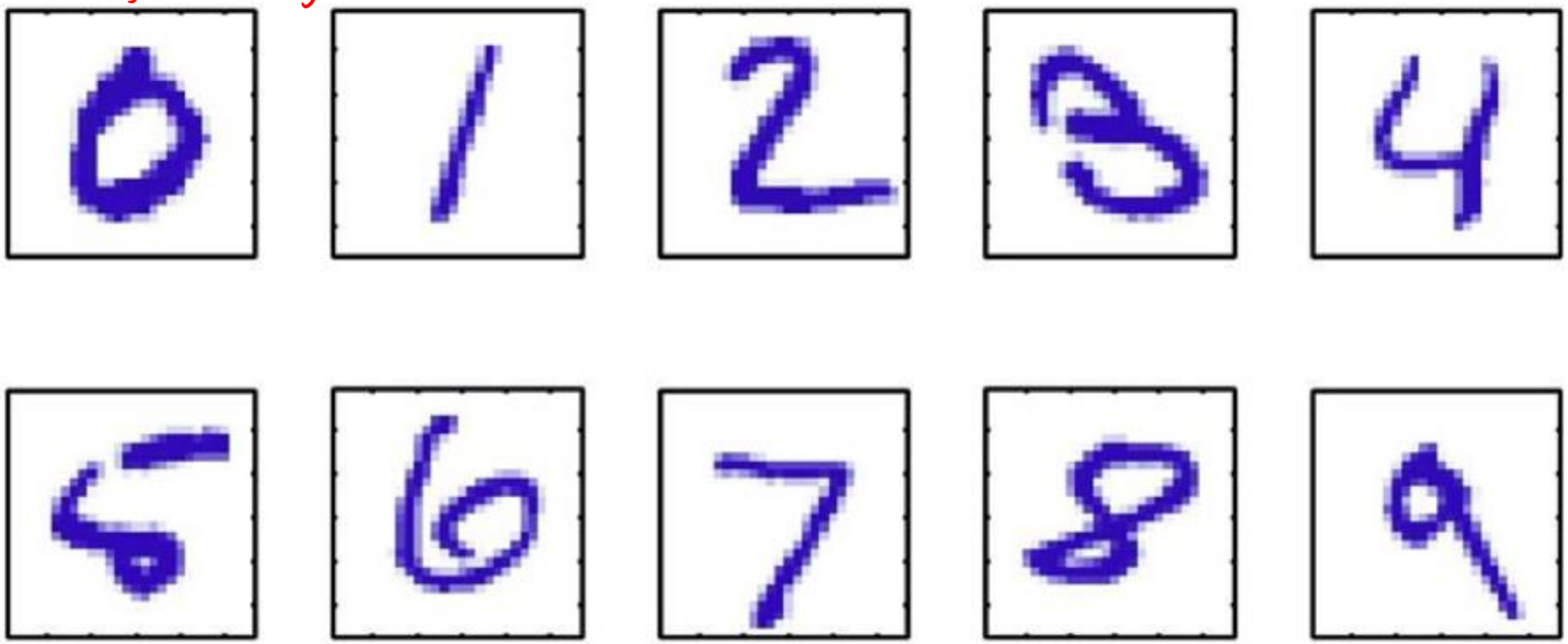
Start with training data, e.g. 6000 examples of each digit



- Can achieve testing error of 0.4%
- One of first commercial and widely used ML systems (for zip codes & checks)

Classification Example 1: Hand-Written Digit Recognition

dimensionality reducing using forward or backward feature selection



Images are 28 x 28 pixels

A classification problem

dimension → 784

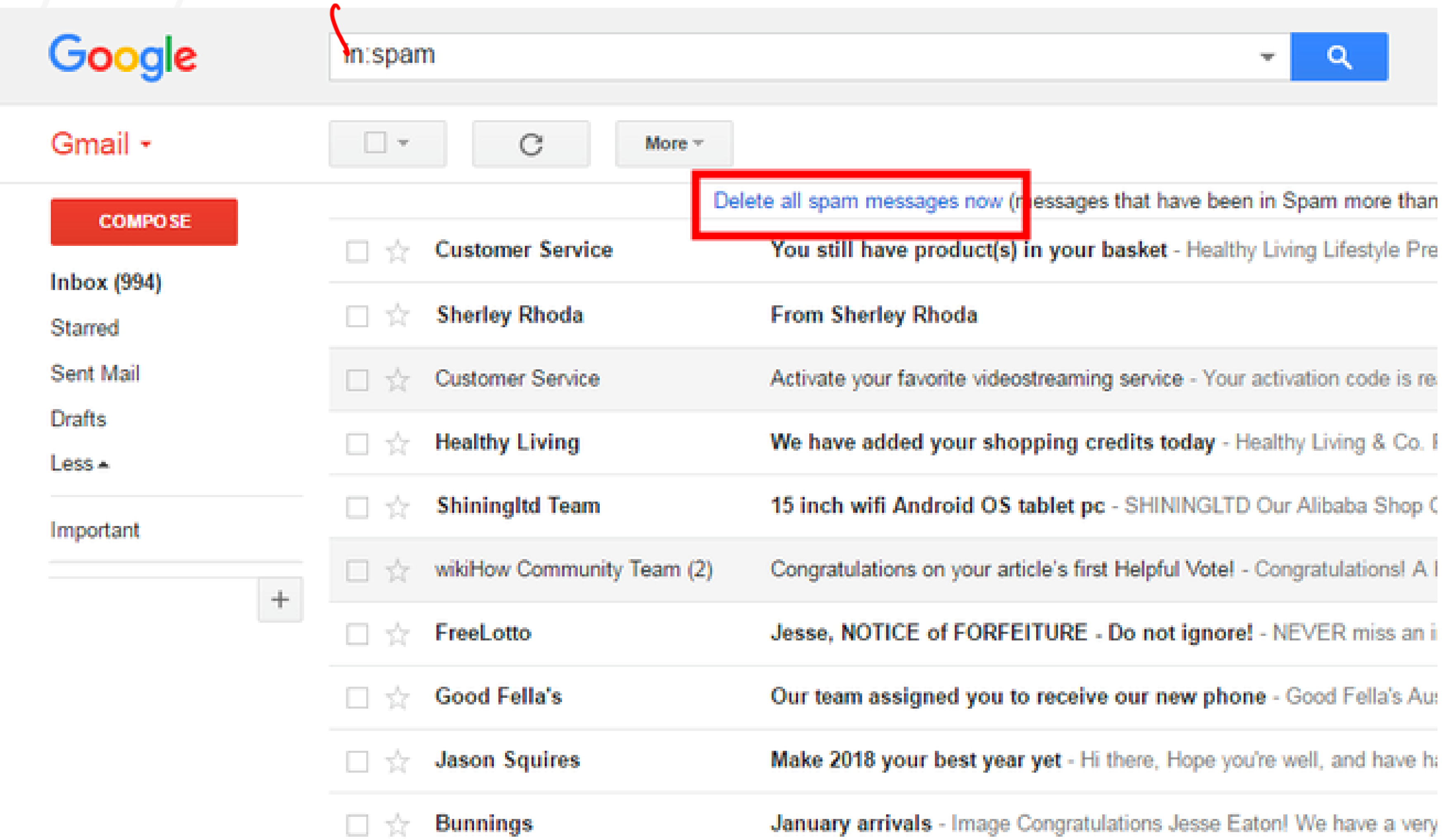
Represent input image as a vector $\mathbf{x} \in \mathbb{R}^{784}$

Learn a classifier $f(\mathbf{x})$ such that,

$$f : \mathbf{x} \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

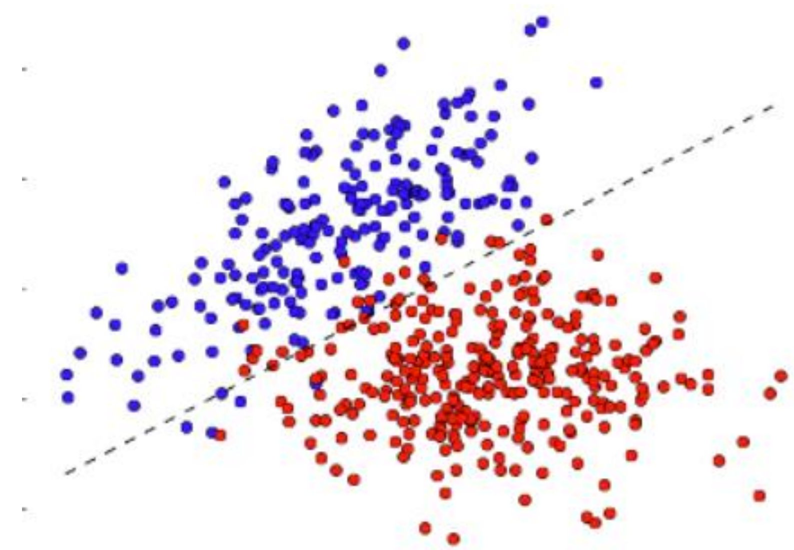
Georgia Tech

Classification Example 2: Spam Detection



① Vocabulary/ Dictionary

NOT SPAM



SPAM

A classification problem

- This is a classification problem
- Task is to classify email into spam/non-spam
- Data x_i is word count
- Requires a learning system as "enemy" keeps innovating

Regression Example 1: Apartment Rent Prediction

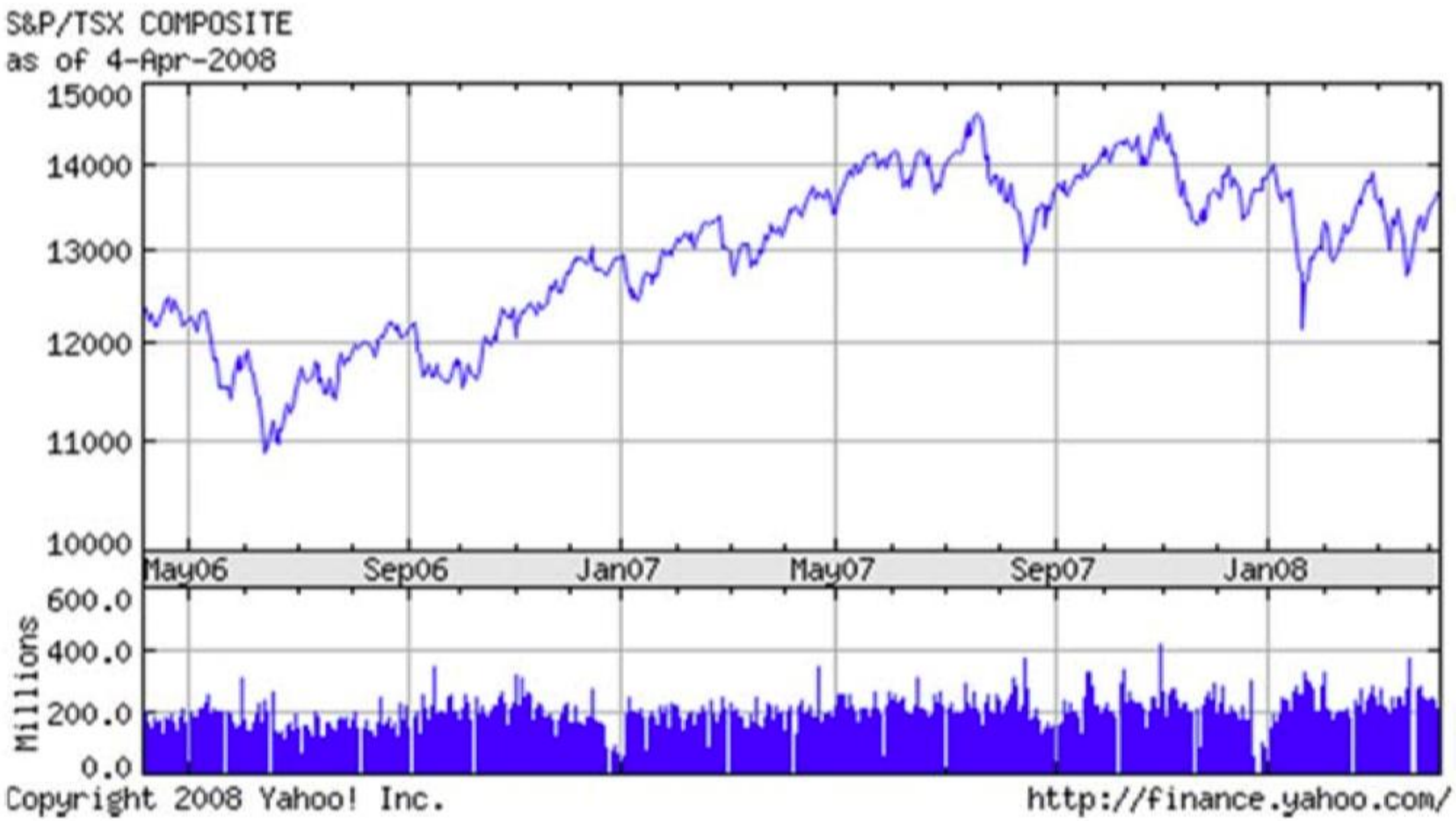
- Suppose you are to move to Atlanta
- And you want to find the **most reasonably priced** apartment satisfying your **needs:**

  square-ft., # of bedroom, distance to campus …

A regression problem

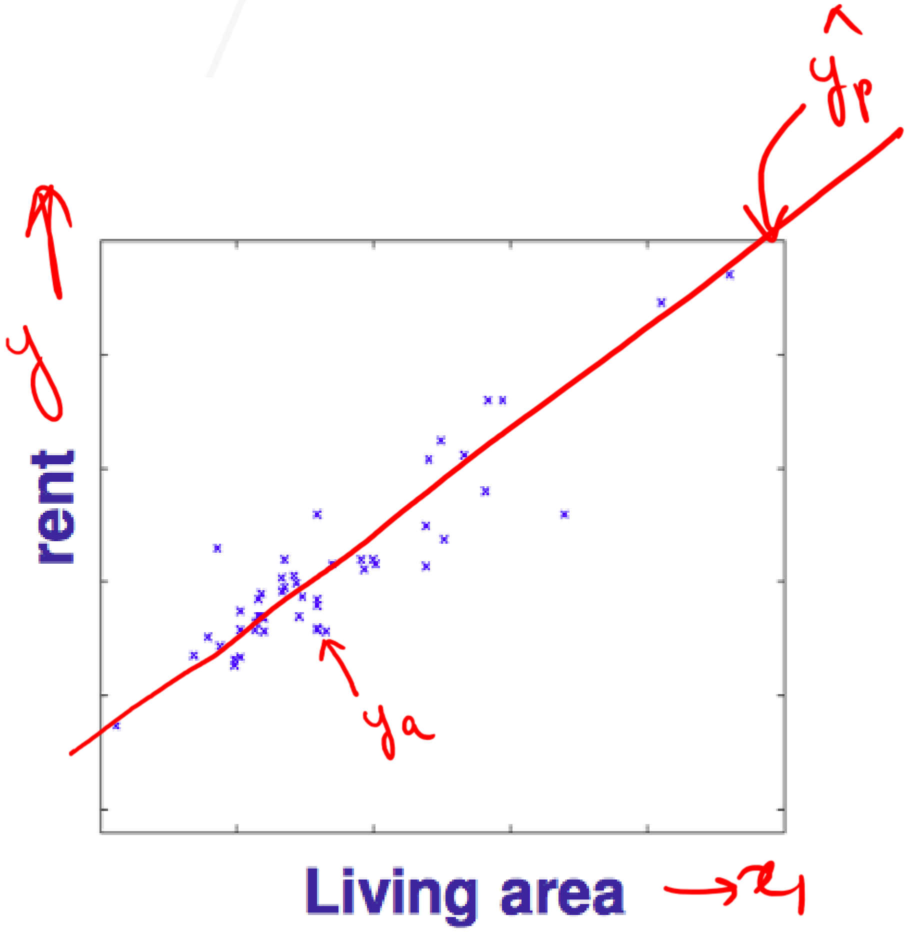
Living area (ft^2)	# bedroom	Rent ($)
230	1	600
506	2	1000
433	2	1100
109	1	500
…		
150	1	?
270	1.5	?

Regression Example 2: Stock Price Prediction



- Task is to predict stock price at future date

A regression problem



- Features:
  - Living area, distance to campus, # bedroom …
  - Denote as $x = (x_1, x_2, \dots, x_d)$
- Target:
  - Rent
  - Denoted as y

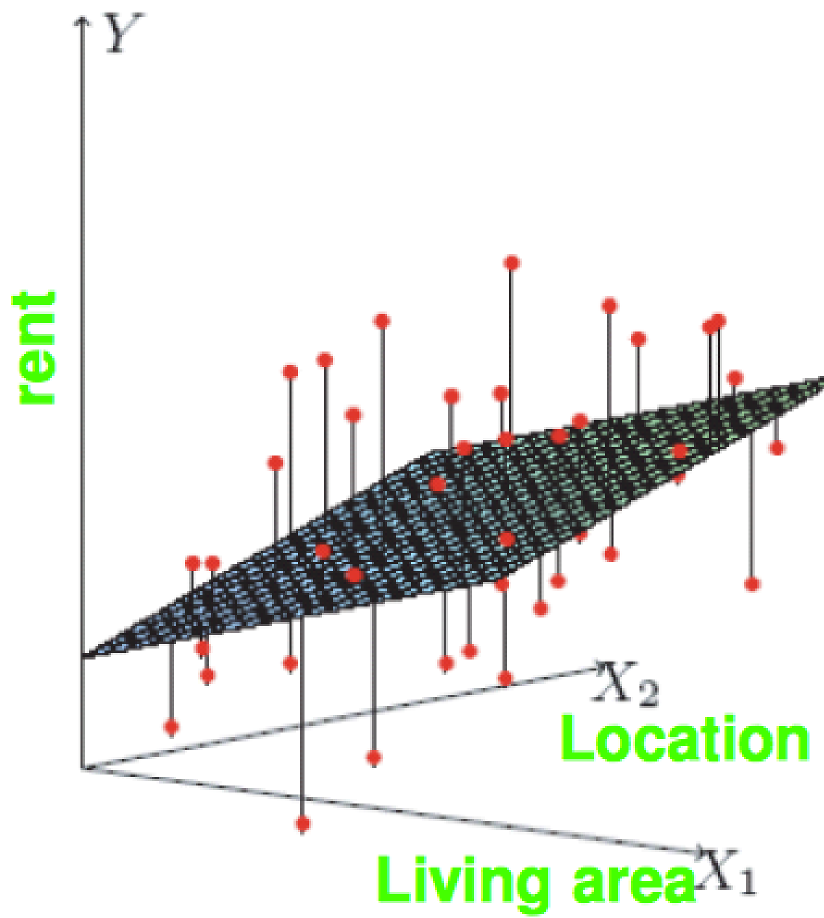
$\hat{y}_p = mx + b$ ← offset or y-intercept

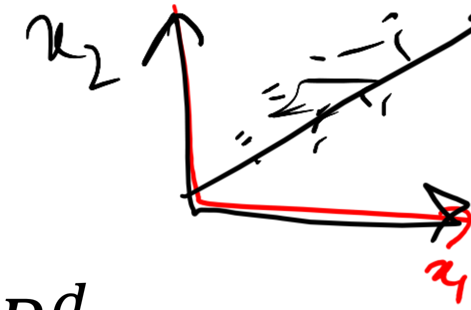
rotation

$\hat{y}_p = \theta_0 + \boxed{\theta_1 x_1}$

bias term

gateways to features





- **Training set:**
  - $x = \{x^{\{1\}}, x^{\{2\}}, \dots, x^{\{n\}}\} \in R^d$
  - $y = \{y^{\{1\}}, y^{\{2\}}, \dots, y^{\{n\}}\}$

$$\hat{y}_p = \theta_0 + \theta_1 \cdot x_1 + \theta_2 x_2$$

d features → (d+1) parameters

$$x^{\{i\}} = \begin{bmatrix} 1 & x_1 & x_2 & x_3 & \dots & x_d \end{bmatrix}$$

(1 × d+1)

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{(d+1)\times 1}$$

$$\boxed{y = \theta \cdot X}$$

LCF ← Linear Combination of Features

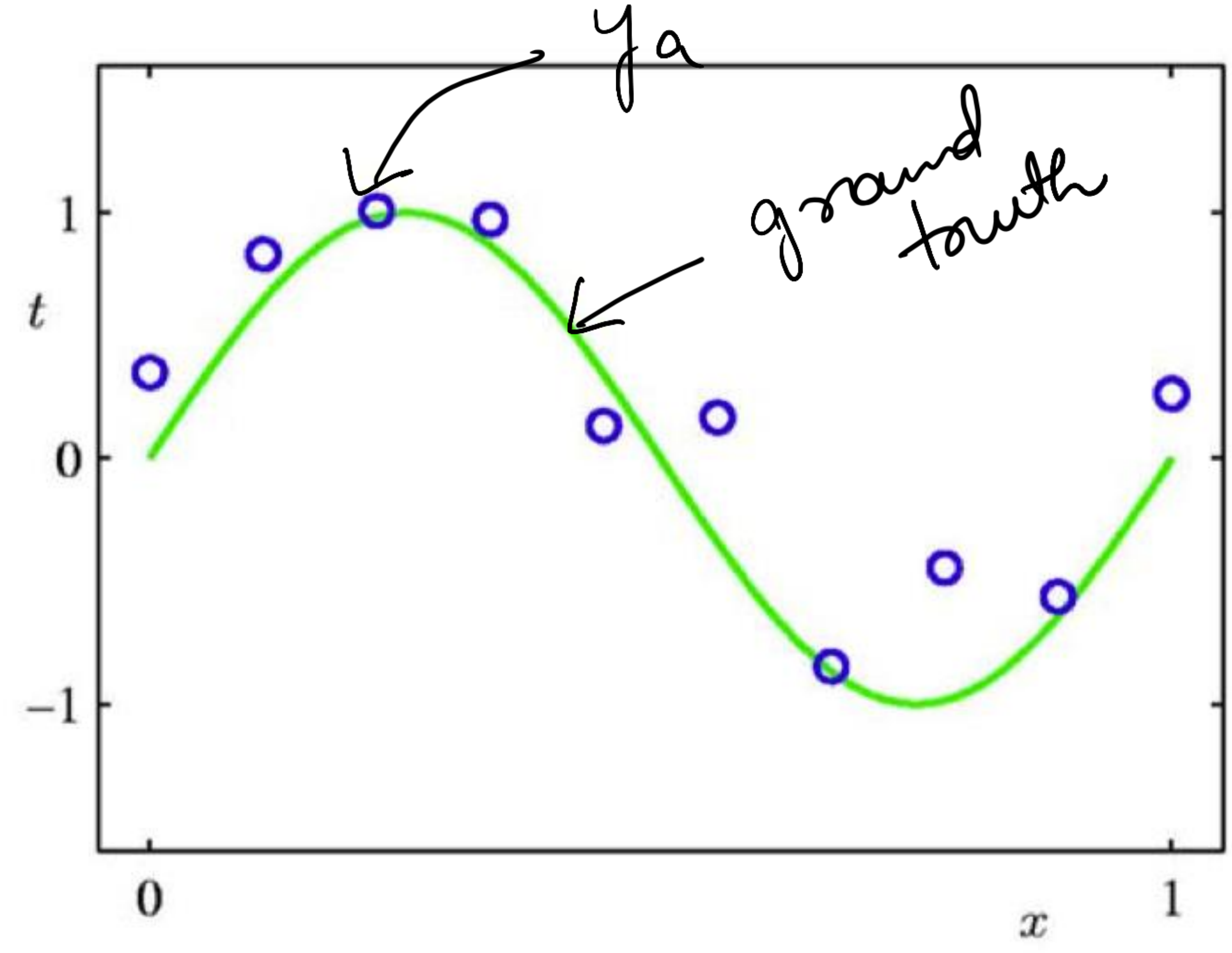
$$y = \theta_0^2 + \theta_1^2 x_1 + \dots + \theta_d^2 x_d$$

X LCF

$$y = \theta_0 + \theta_1 x_1^2 + \theta_2 x_2^2 + \dots + \theta_d x_d^2$$

← LCF

Regression: Problem Setup



- Suppose we are given a training set of N observations

- Regression problem is to estimate y(x) from this data

Outline

- Supervised Learning
- Linear Regression
- Extension

Linear Regression

- Assume y is a linear function of x (features) plus noise ϵ

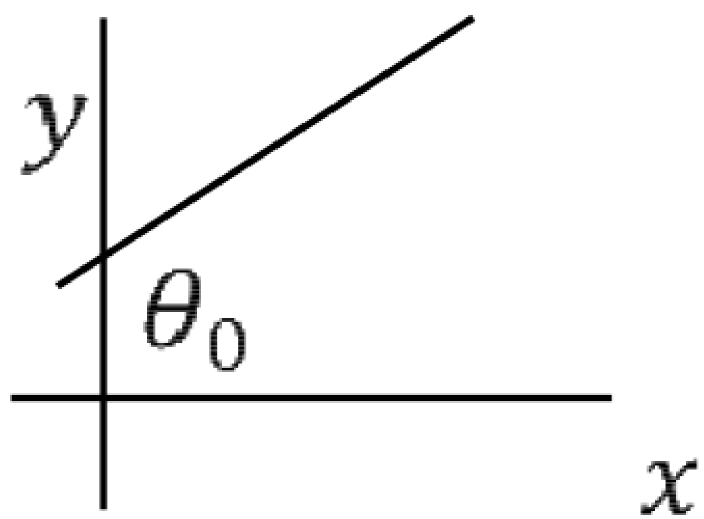
$$y = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d + \epsilon$$

noise

- where ϵ is an error term of unmodeled effects or random noise

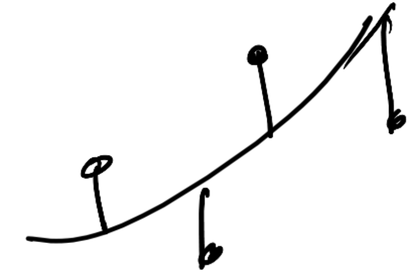
- Let $\theta = (\theta_0, \theta_1, \ldots, \theta_d)^\top$, and augment data by one dimension

- Then $y = x\theta + \epsilon$



Least Mean Square Method

$\|y^{\{i\}} - x^{\{i\}}\theta\|$ $E[\theta]$



- Given n data points, find θ that minimizes the mean square error

Training

$$\hat{\theta} = argmin_{\theta}\ \underbrace{L(\theta)}_{E(\theta)} = \frac{1}{n}\sum_{i=1}^{N}\Big(\underbrace{\overset{1\times 1}{y^{\{i\}}}}_{y_a} - \underbrace{\overset{1\times(d+1)}{x^{\{i\}}}\overset{(d+1)\times 1}{\theta}}_{\hat{y}_p}\Big)^2$$

- Our usual trick: set gradient to 0 and find parameter

$$\frac{\partial L(\theta)}{\partial \theta} = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\sum_{i=1}^{N}(x^{\{i\}})^T\,(y^{\{i\}} - x^{\{i\}}\theta) = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\sum_{i=1}^{N}(x^{\{i\}})^T y^{\{i\}} + \frac{2}{n}\sum_{i=1}^{N}(x^{\{i\}})^T x^{\{i\}}\theta = 0$$

Matrix form

$$x = \begin{bmatrix} 1 & x_1^{\{1\}} & \cdots & x_d^{\{1\}} \\ 1 & x_1^{\{2\}} & \ddots & x_d^{\{2\}} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{\{n\}} & \cdots & x_d^{\{n\}} \end{bmatrix}_{n \times (d+1)} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{(d+1) \times 1}$$

$$MSE(\theta) = argmin_{\,\theta}\, L(\theta) = \frac{1}{n}(y - x\theta)^{\mathrm{T}}(\mathrm{y} - x\theta)$$

$$x\theta = \begin{bmatrix} \theta_0 + \theta_1 x_1^{\{1\}} + \theta_2 x_2^{\{1\}} + \cdots + \theta_d x_d^{\{1\}} \\ \theta_0 + \theta_1 x_1^{\{2\}} + \theta_2 x_2^{\{2\}} + \cdots + \theta_d x_d^{\{2\}} \\ \vdots \\ \theta_0 + \theta_1 x_1^{\{n\}} + \theta_2 x_2^{\{n\}} + \cdots + \theta_d x_d^{\{n\}} \end{bmatrix}_{n \times 1}$$

$y = X\theta.$

$\theta = X^{-1} \cdot y$

Matrix Version and Optimization

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\boxed{\sum_{i=1}^{N} (x^{\{i\}})^T y^{\{i\}}} + \frac{2}{n}\boxed{\sum_{i=1}^{N} (x^{\{i\}})^T x^{\{i\}} \theta} = 0$$

Let’s rewrite it as:

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\left(x^{\{1\}}, \dots, x^{\{n\}}\right)^T \left(y^{\{1\}}, \dots, y^{\{n\}}\right) + \frac{2}{n}\left(x^{\{1\}}, \dots, x^{\{n\}}\right)^T \left(x^{\{1\}}, \dots, x^{\{n\}}\right)\theta = 0$$

$$Define\, \mathrm{X} = \left(x^{\{1\}}, \dots, x^{\{n\}}\right)\ and\, \mathrm{y} = \left(y^{\{1\}}, \dots, y^{\{n\}}\right)$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\mathrm{X}^{\mathrm{T}}\mathrm{y} + \frac{2}{n}X^T X\theta = 0$$

$$\Rightarrow \theta = (X^T X)^{-1} X^T y = \boxed{X^+ y}$$

X^+ *is the pseudo-inverse of* X

$$X^T X X^+ = X^T$$

$$\theta = (X^T X)^{-1} X^T y = X^+ y$$

$X_{n \times d}$ $\quad$ n = instances $\quad$ d = dimension

$$X^T X = \begin{bmatrix} d \times n \end{bmatrix} \begin{bmatrix} n \times d \end{bmatrix} = \begin{bmatrix} d \times d \end{bmatrix}$$

Not a big matrix because $n \gg d$ *This matrix is invertible most of the times. If we are VERY unlucky and columns of* $\mathbf{X}^{\mathbf{T}}\mathbf{X}$ *are not linearly independent (it's not a full rank matrix), then it is not invertible.*

Quick Knowledge Check

1. What is the main difference between regression and classification tasks? A. Regression predicts discrete labels, classification predicts continuous outputs. B. Regression predicts continuous values, classification predicts discrete labels. C. Both predict continuous values. D. Both predict categories
2. Which of the following is an example of a regression task? A. Detecting spam emails. B. Predicting a car's fuel efficiency. C. Recognizing handwritten digits. D. Detecting positive vs. negative sentiments in reviews
3. In **backward feature selection**, what happens at each step? A. The feature that least improves performance is added. B. The feature that least reduces performance is removed. C. The feature with the lowest correlation is removed. D. The most important feature is added
4. In the linear regression equation $\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$,what does **b** represent? A. The slope of the model. B. The learning rate. C. The bias or intercept term. D. The regularization penalty $\leftarrow \epsilon$
5. What is the goal of the least mean square (LMS) method? A. Minimize classification errors. B. Maximize model complexity. C. Minimize the sum of squared residuals between predictions and actual values. D. Minimize correlation between features
6. Why is it important to include a bias term in linear regression? A. To allow the model to fit data that doesn't pass through the origin. B. To normalize input features. C. To regularize the model. D. To reduce computational cost

Recap

Linear Combination of Features

$$\hat{y}_p = \theta . X = \theta_0 + \theta_1 X_1 + \theta_2 X_2 \ldots\ldots + \theta_d X_d .$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{(d+1) \times 1} \qquad X^{\{i\}} = [1 \; x_1 \; x_2 \ldots x_d]_{1 \times (d+1)}$$

$$X^{\{i\}} . \theta = \text{scalar} = \hat{y}_p .$$

- LCF
- Objective function

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(y_a^{\{i\}} - X^{\{i\}} . \theta \right)^2 \leftarrow \text{Convex.} \qquad X_{n \times d+1} \quad X^T_{d+1 \times n}$$

$E(\theta)$

$$\underset{(d+1) \times 1}{\theta} = X^{+} Y = \boxed{\underset{(d+1)(d+1)}{(X^T X)^{-1}} . \; X^T . Y}$$

Global sol.

- Global solution
- Why good? a) Global sol. b) Closed form Sol
- Why bad? a) Inverse b) $(X^T X) \leftarrow$ supremely expensive

Alternative Way to Optimize

$y = x^2$

$\frac{dy}{dx} = 0$

$x = 0$

Global sol. $\rightarrow x = 0$

$$x^{(t+1)} = x^{(t)} - \alpha \cdot \frac{\partial y}{\partial x}$$

- The matrix inversion in $\hat{\theta} = (X^T X)^{-1} X^T y$ can be very expensive to compute

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^{N} (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}}\theta)$$

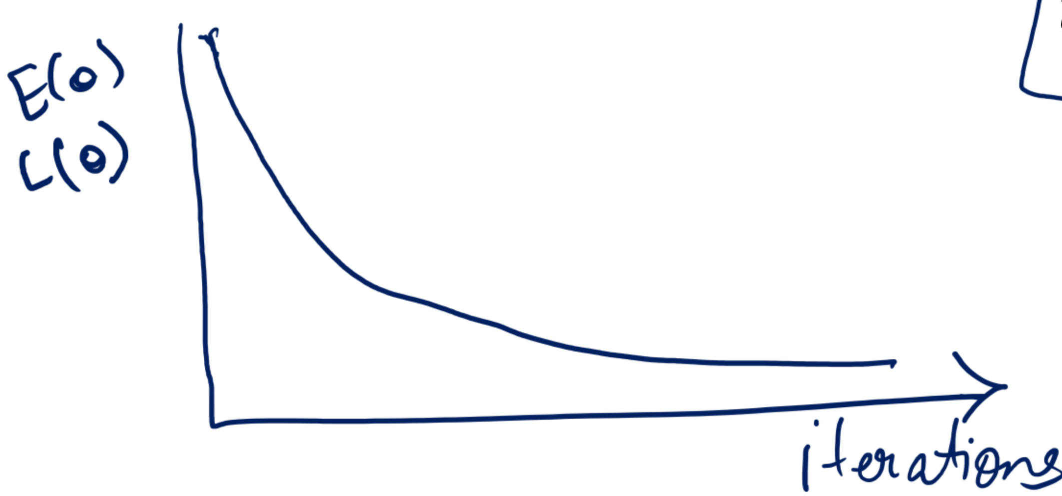
$$\theta^{t+1} = \theta^{t} - \alpha \cdot \frac{\partial L}{\partial \theta}$$

hyperparameter

- Gradient descent

GD or FGD

$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^{t} + \frac{\alpha}{n} \sum_{i=1}^{N} (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}}\theta)$$



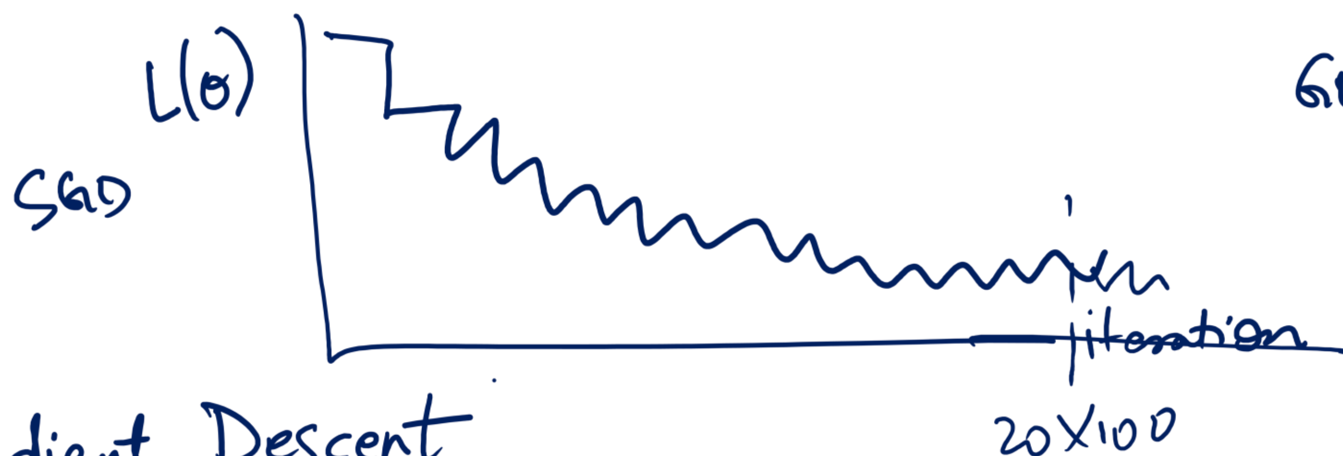
Alternative Way to Optimize

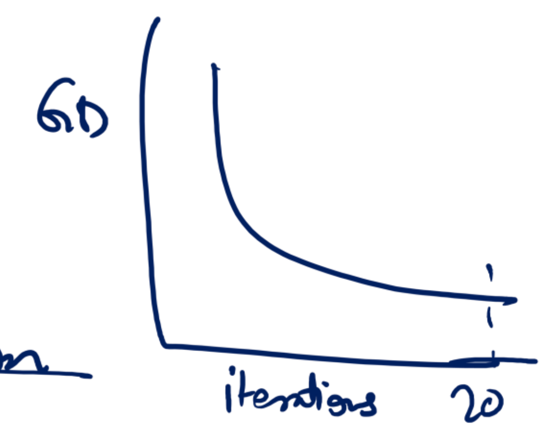
SGD

- Stochastic gradient descent (use one data point at a time) 100 datapoints

$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^{t} + \beta_t \times (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}}\theta)$$

Updating θ very quickly
Overall convergence may be slow

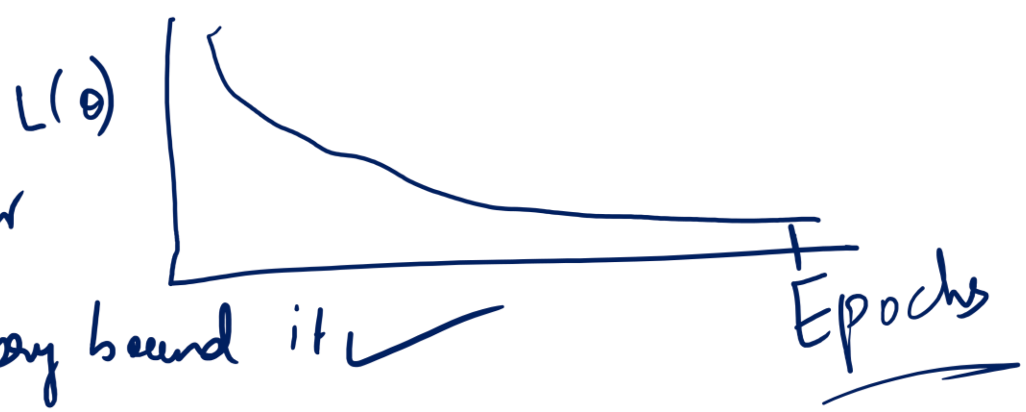




BGD ← Batch Gradient Descent

[X] ← [ ] [ ] [ ] [ ] [ ]

no. of batches
↓
hyper parameter
↳ memory bound it



Epoch ← 1 epoch means we have gone through all datapoints

Recap

- Stochastic gradient update rule ← 1 datapoint at a time

$$\hat{\theta}^{t+1} \leftarrow \hat{\theta}^{t} + \beta \times (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}}\theta)$$

$\boxed{\text{BGD}}$

  - Pros: on-line, low per-step cost
  - Cons: coordinate, maybe slow-converging
- Gradient descent ← all datapoints

$$\hat{\theta}^{t+1} \leftarrow + \frac{\alpha}{n} \sum_{i=1}^{N} (x^{\{i\}})^T (y^{\{i\}} - x^{\{i\}}\theta)$$

  - Pros: fast-converging, easy to implement
  - Cons: need to read all data
- Solve normal equations

$$\theta = (X^T X)^{-1} X^T y$$

  - Pros: a single-shot algorithm! Easiest to implement.
  - Cons: need to compute inverse $(X^T X)^{-1}$, expensive, numerical issues (e.g., matrix is singular ..)

Linear regression for classification

features $\rightarrow$ 256
$\theta \rightarrow 257$

Raw Input $x = (1, x_1, \ldots, x_{256})$

Linear model $(\theta_0, \theta_1, \ldots, \theta_{256})$

16

16

Extract useful information

$$\underbrace{\text{intensity and symmetry}}_{\text{feature engineering}}\ \theta = (1, x_1, x_2)$$

Sum up all the pixels = intensity

Symmetry = -(difference between flip version)

$$\theta = (\overset{\theta_0}{1}, x_1, x_2)$$

$$x_1 = intensity \quad x_2 = symmetry$$

$\hat{y}_p = -510$

$\hat{y}_p = +71.46$

$\hat{y}_p = 0$

$\boxed{\text{sign}}$

Encode

↓

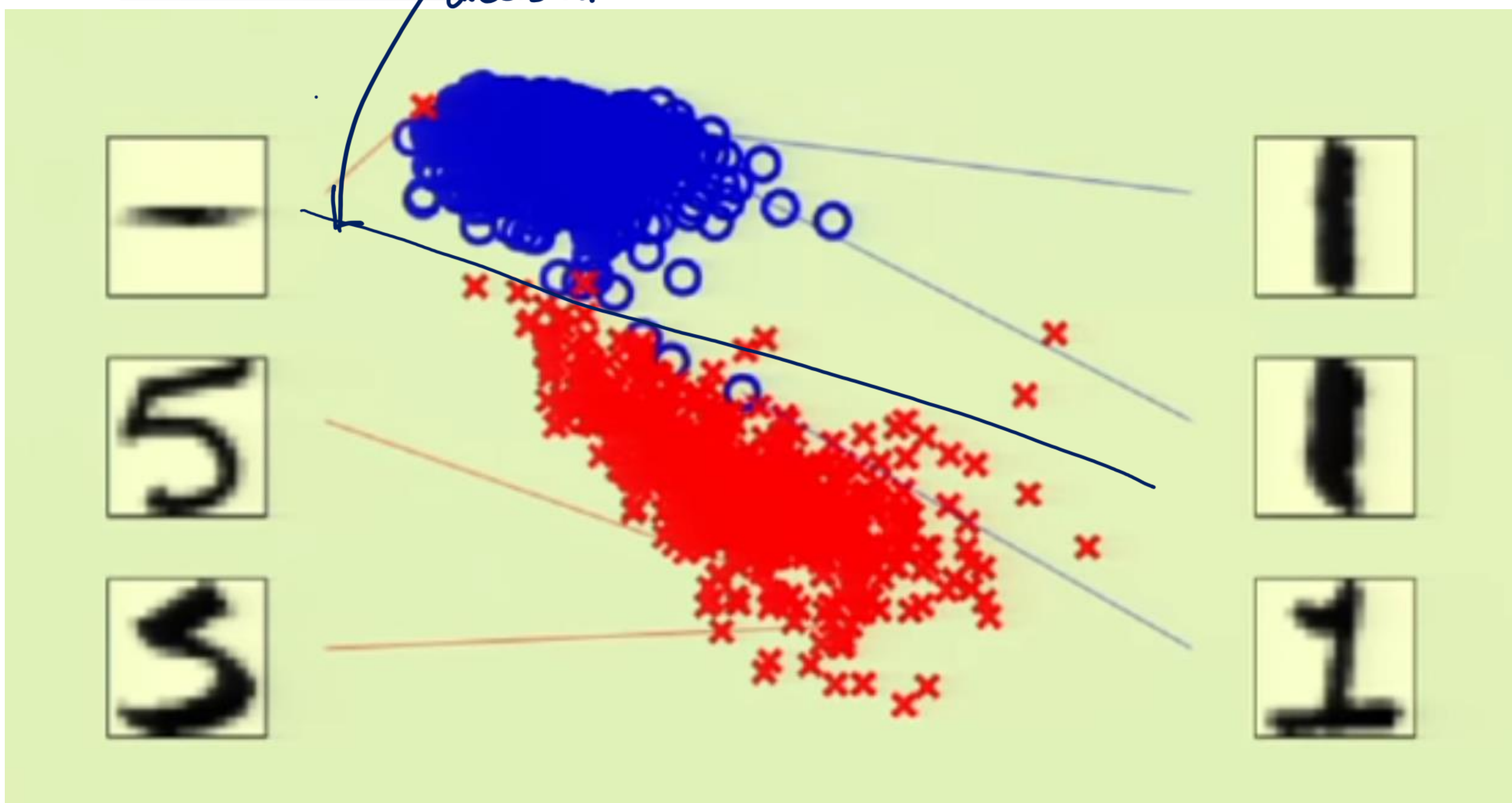
−1 output 5

It is almost linearly separable

1 output 1

decision line

symmetry



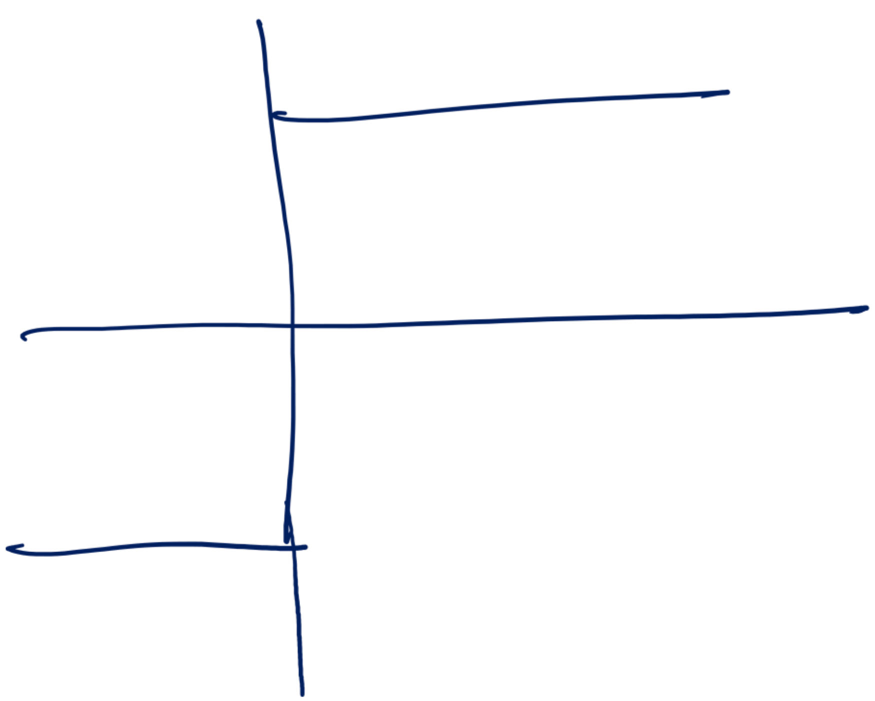
intensity

Linear regression for classification

Binary-valued functions are also real-valued $\pm 1 \in R$

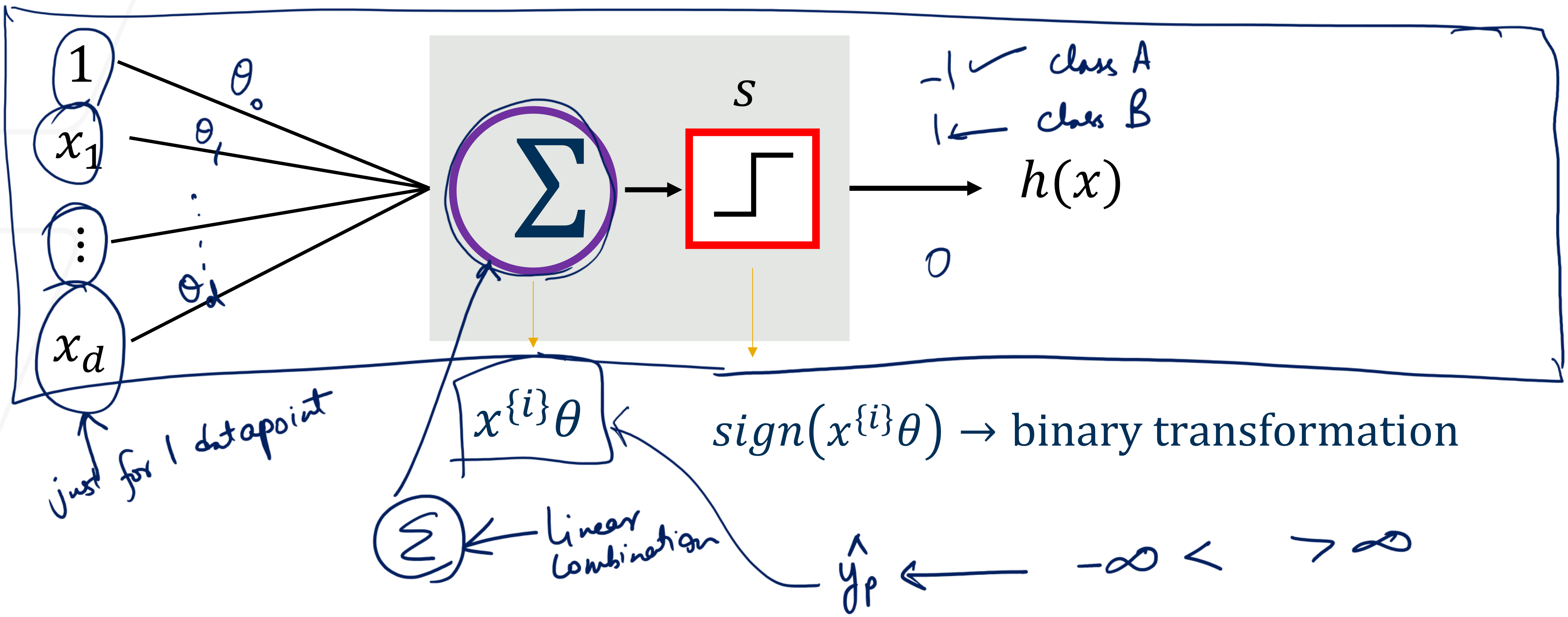
Use linear regression $\overset{\hat{y}_p}{x^{\{i\}}\theta} \approx \overset{y_a}{y^{\{i\}}} = \pm 1$ $\quad i = index\ of\ a\ data\text{-}point$

Let's calculate, $sign\left(x^{\{i\}}\theta\right) = \begin{cases} -1 & x^{\{i\}}\theta < 0 \\ 0 & x^{\{i\}}\theta = 0 \\ 1 & x^{\{i\}}\theta > 0 \end{cases}$

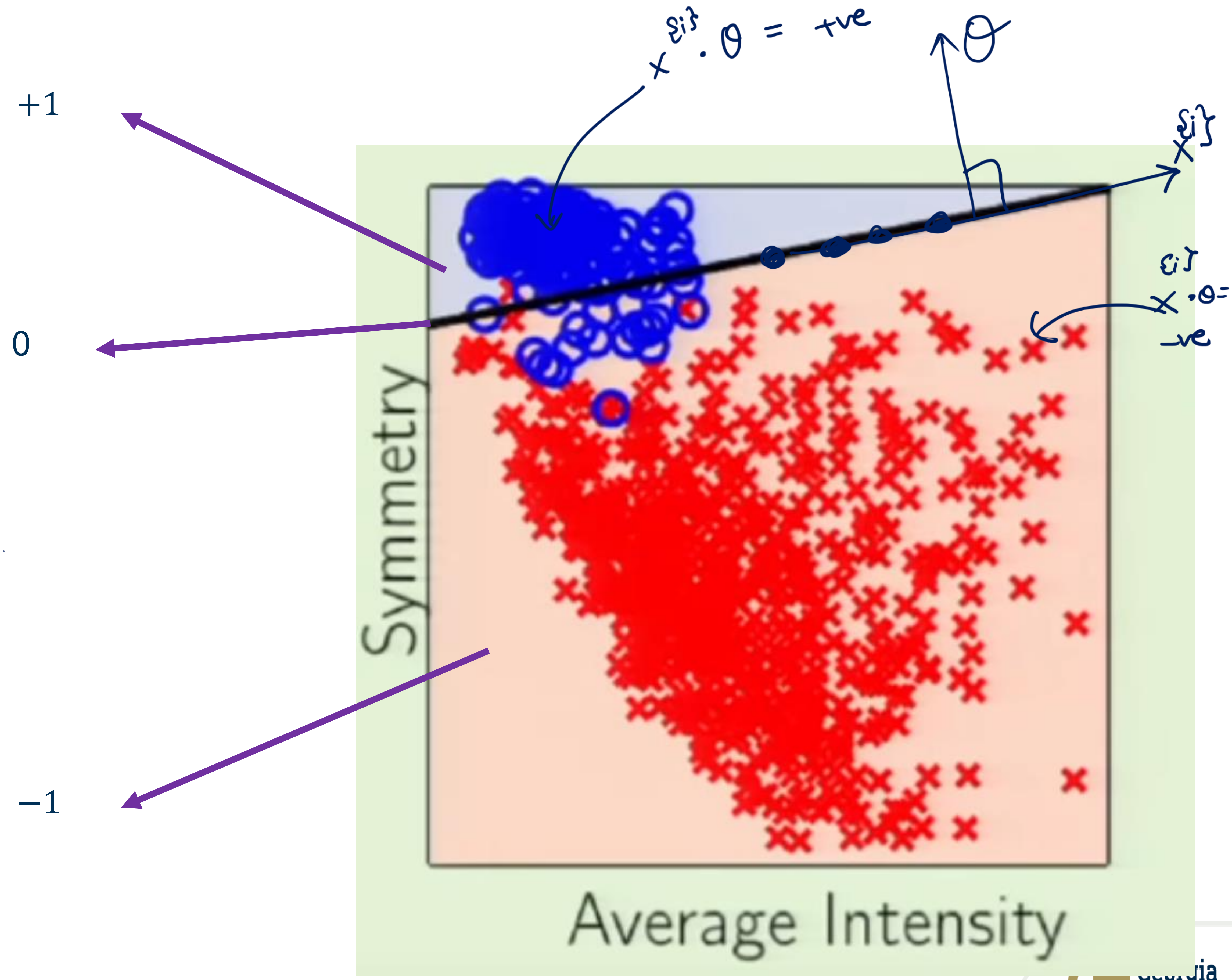


Linear regression for classification

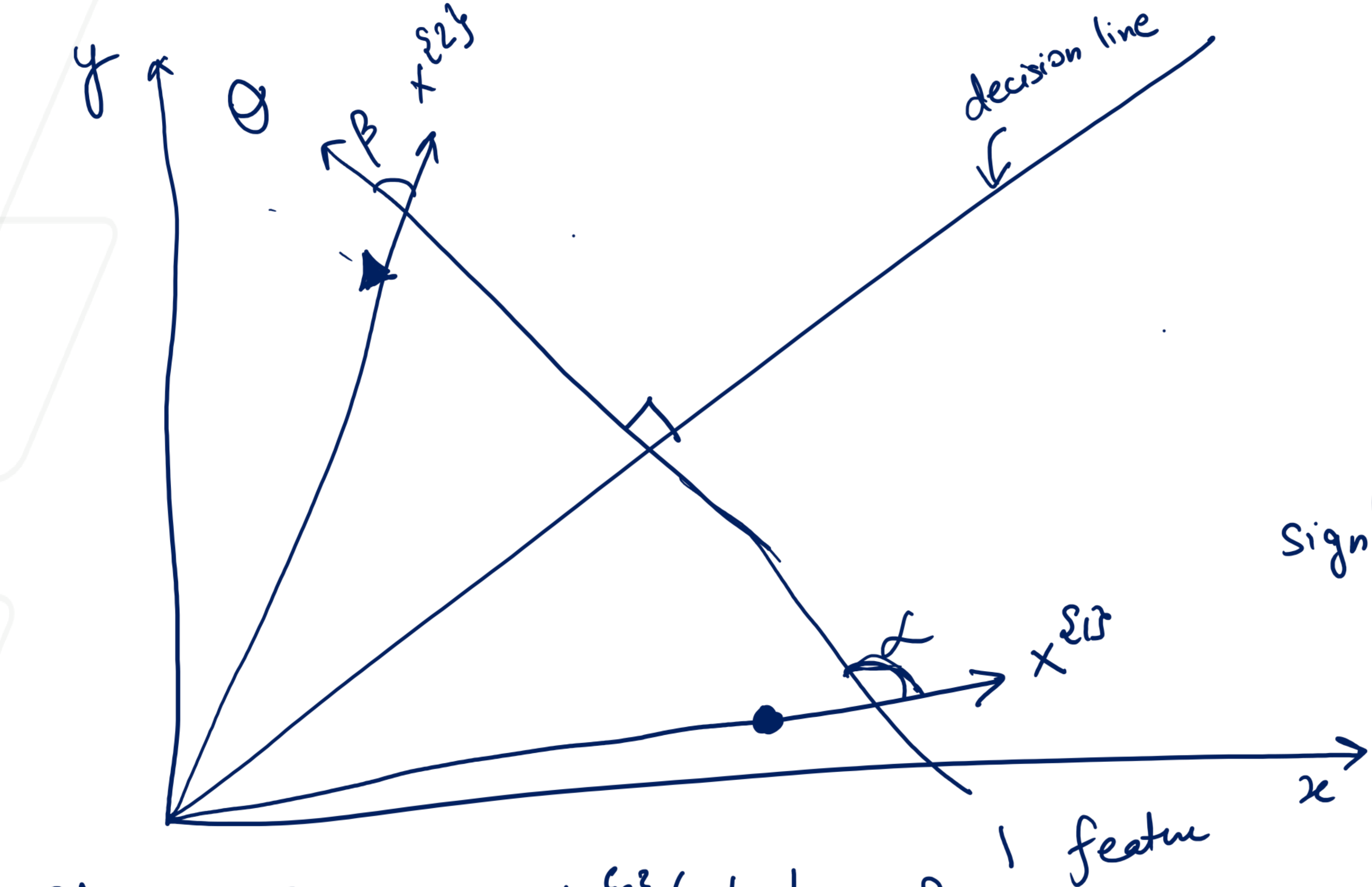
SGD For one data point (data-point i) with **d** dimensions (instance):



$sign(x^{\{i\}}\theta) \rightarrow$ binary transformation



Not really the best for classification, but it's a good start



$$\hat{y}_p^{\{1\}} = X^{\{1\}} \cdot \theta$$

$$= |X^{\{1\}}| \cdot |\theta| \cdot \cos\alpha$$

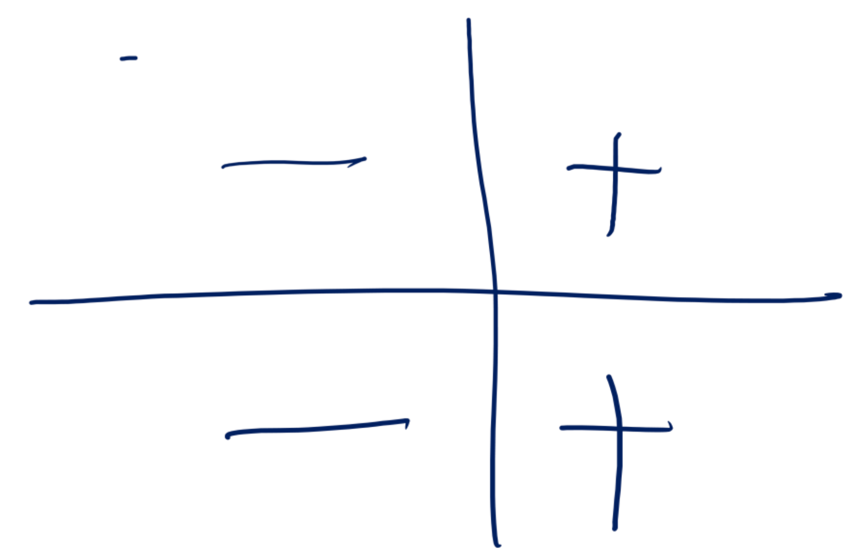
$$= \quad +ve \quad +ve \quad -ve$$

$$\text{sign}\left(\hat{y}_p^{\{1\}}\right) = -1$$

$$\hat{y}_p^{\{2\}} = X^{\{2\}} \cdot \theta = |X^{\{2\}}| \cdot |\theta| \cdot \cos\beta$$

$$+ve \quad +ve \quad +ve$$

$$\text{sign}\left(\hat{y}_p^{\{2\}}\right) = +1$$



https://www.math3d.org/UPNV1IQvxt

Why Linear Regression is not good for classification

1. Output Range Problem

Linear regression produces continuous outputs from $-\infty$to $+\infty$. But in classification (e.g., predicting 0 or 1), you need probabilities — values between 0 and 1. If you try to interpret the linear output as a probability, you'll often get nonsensical values (like 1.3 or -0.5).

2. Poor Decision Boundary for Binary Classes

Suppose you define a rule like "if prediction > 0.5 → class 1, else → class 0." Linear regression will fit a line minimizing squared error, not a boundary that best separates classes. That means outliers or overlapping regions can drastically skew the boundary. In short: it's not optimizing for classification accuracy, but for minimizing distance between predicted and actual numeric labels (0 or 1).

3. Sensitivity to Outliers

Because linear regression uses mean squared error (MSE), large errors (caused by mislabeled or extreme data points) have a big impact. This distorts the model's predictions and decision line.

4. Non-linearity of Class Boundaries

Many classification problems have non-linear separations between classes. Linear regression can only produce a straight-line boundary, so it fails when data isn't linearly separable.

Georgia Tech

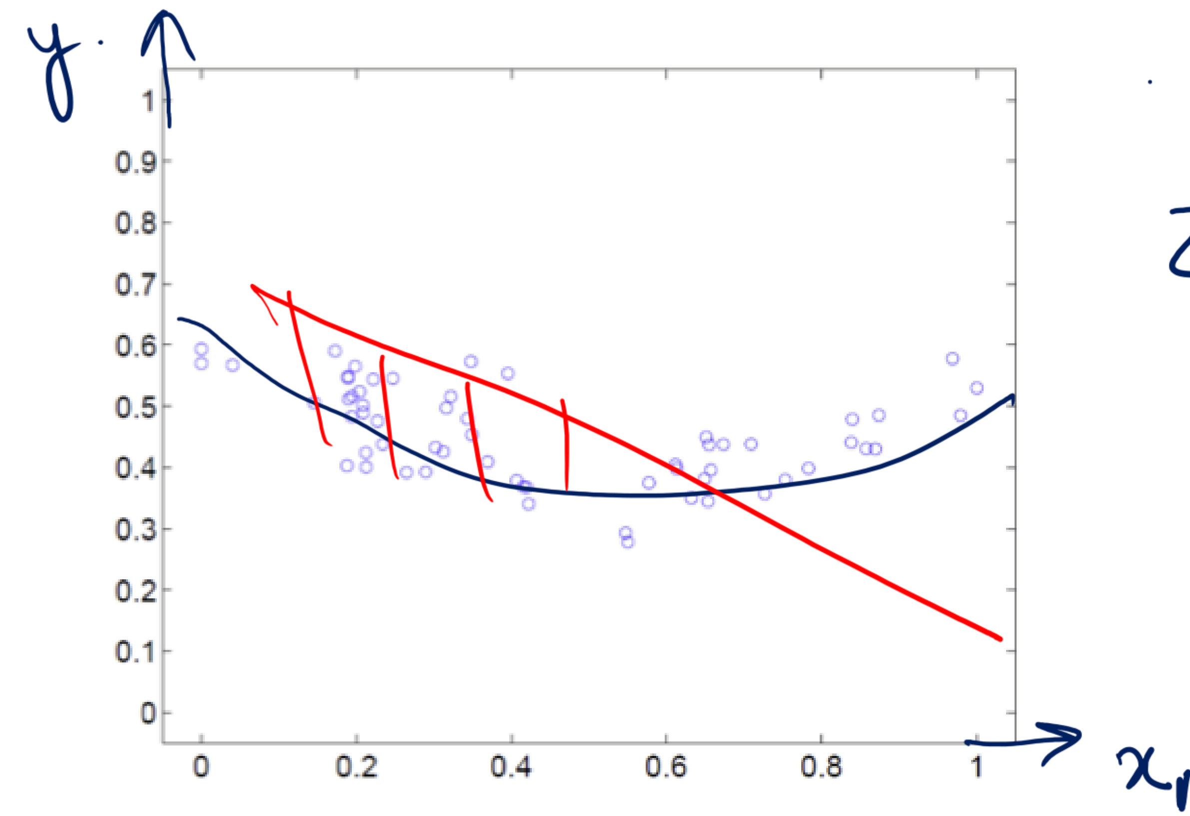
Outline

- Supervised Learning
- Linear Regression
- Extension

Extension to Higher-Order Regression

Feature Engineering

$$x = \begin{bmatrix} x_1 \end{bmatrix}$$



$$Z = \begin{bmatrix} x_1 & x_1^2 \\ & \end{bmatrix}$$

- Want to fit a polynomial regression model

LCF

$$\left. y = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d + \epsilon \right\} \text{z space}$$

- $z = \{1, x, x^2, \ldots, x^d\} \in R^d$ and $\theta = (\theta_0, \theta_1, \theta_2, \ldots, \theta_d)^T$

$$\boxed{y = z\theta}$$

z space we have d features $(x \quad x^2 \quad x^3 \quad \cdots \quad x^d)$

x space we have 1 feature

$X \longrightarrow Z$

More features $\qquad d$

↓

More θ $\qquad d+1$

↓

More complex model

↓

More time / More memory

↓

Higher Risk for Overfitting

Least Mean Square Still Works the Same

- Given n data points, find θ that minimizes the mean square error

$$\hat{\theta} = argmin_{\theta}\, L(\theta) = \frac{1}{n}\sum_{i=1}^{N}\left(y^{\{i\}} - z^{\{i\}}\theta\right)^2$$

- Our usual trick: set gradient to 0 and find parameter

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\sum_{i=1}^{N}(z^{\{i\}})^T(y^{\{i\}} - z^{\{i\}}\theta) = 0$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n}\sum_{i=1}^{N}(z^{\{i\}})^T y^{\{i\}} + \frac{2}{n}\sum_{i=1}^{N}(z^{\{i\}})^T z^{\{i\}}\theta = 0$$

Matrix Version of the Gradient

$$z = \{1, x, x^2, \dots, x^d\} \in R^d \qquad y = \{y^{\{1\}}, y^{\{2\}}, \dots, y^{\{n\}}\}$$

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} z^{\mathrm{T}} \mathrm{y} + \frac{2}{n} z^T z\theta = 0$$

$$\Rightarrow \theta = (z^T z)^{-1} z^T y = z^+ y$$

- If we choose a different maximal degree **d** for the polynomial, the solution will be different.

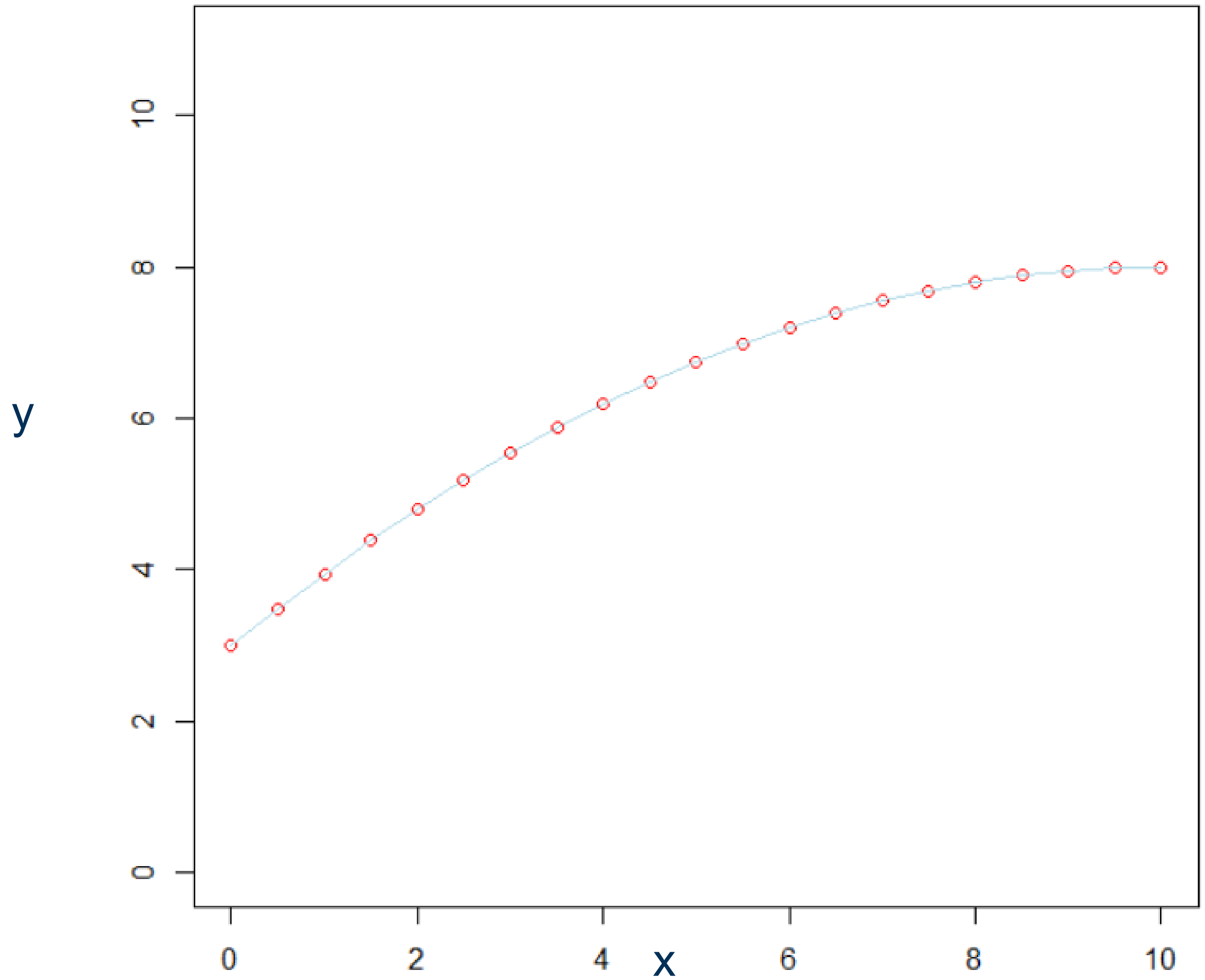
What is happening in polynomial regression?

$x = [0,0.5,1,\ldots,9.5,10]$

$y = [3,3.4875,3.95,\ldots,7.98,8]$

$$f = \theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_0 = 3; \theta_1 = 1; \theta_2 = -0.5$$

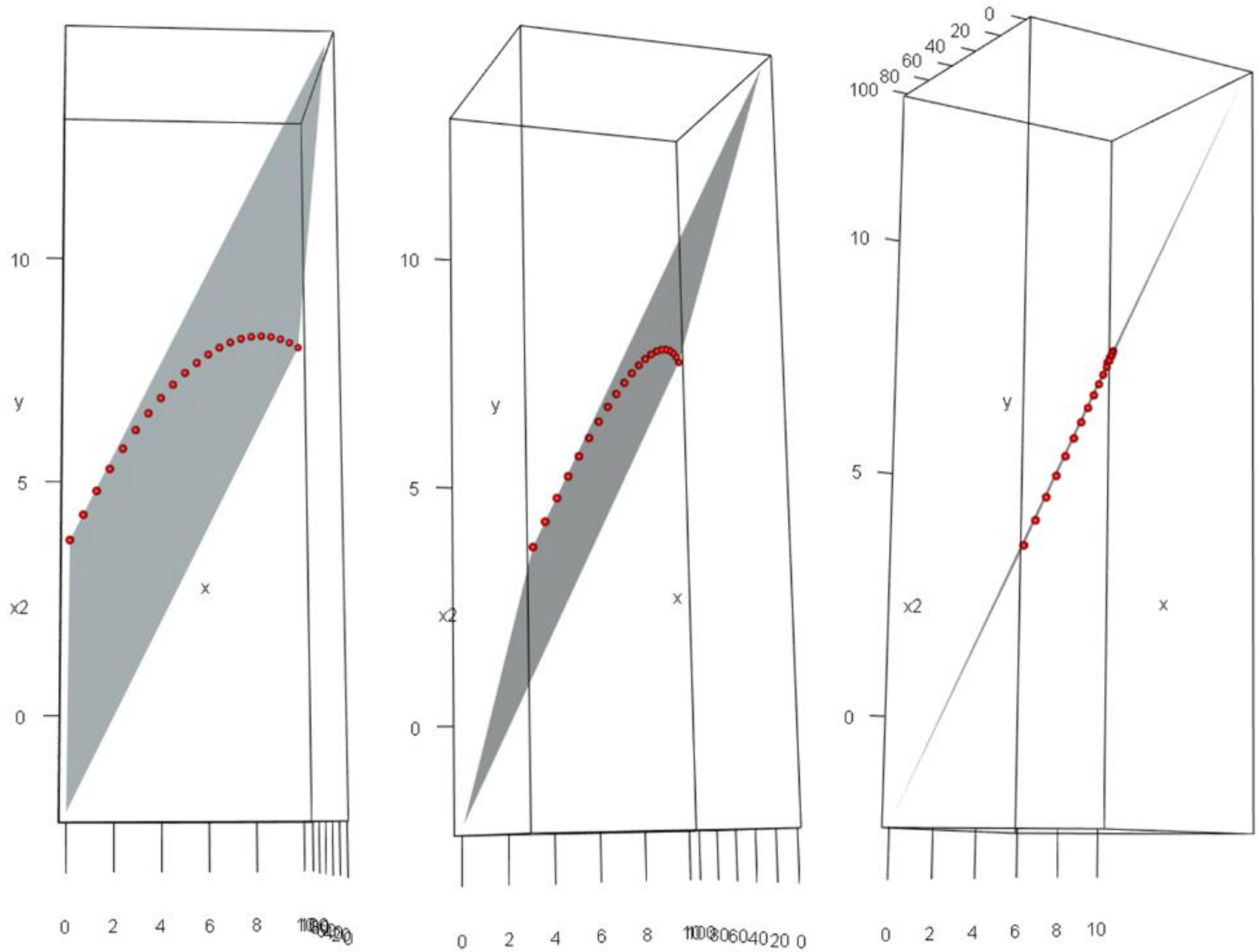


RMSE=0

Let's add to the feature space

$x_1 = [0,0.5,1,\ldots,9.5,10]$ $\quad$ $x_2^2 = [0,0.25,1,\ldots,90.25,100]$

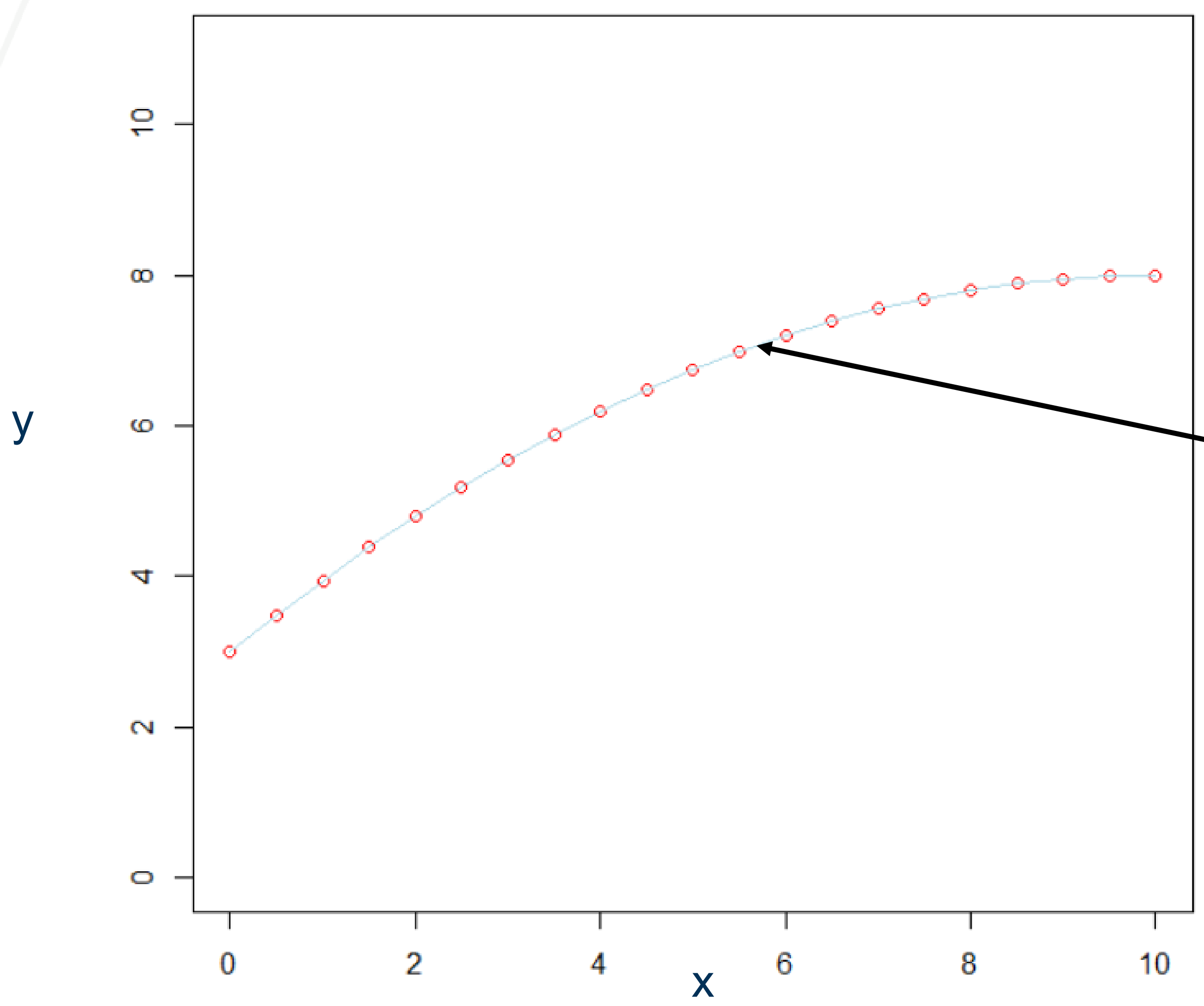
$$y = [3,3.4875,3.95,\ldots,7.98,8]$$

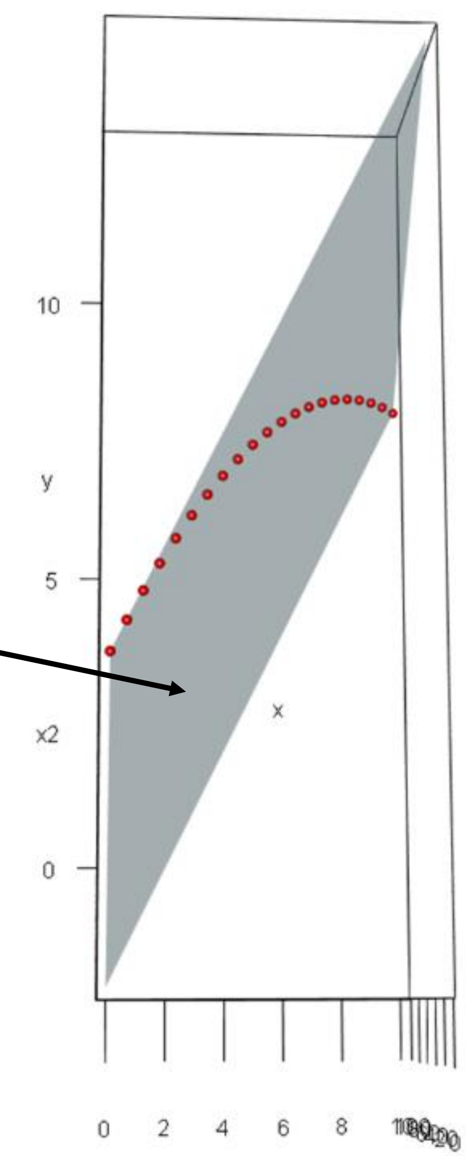


y_p

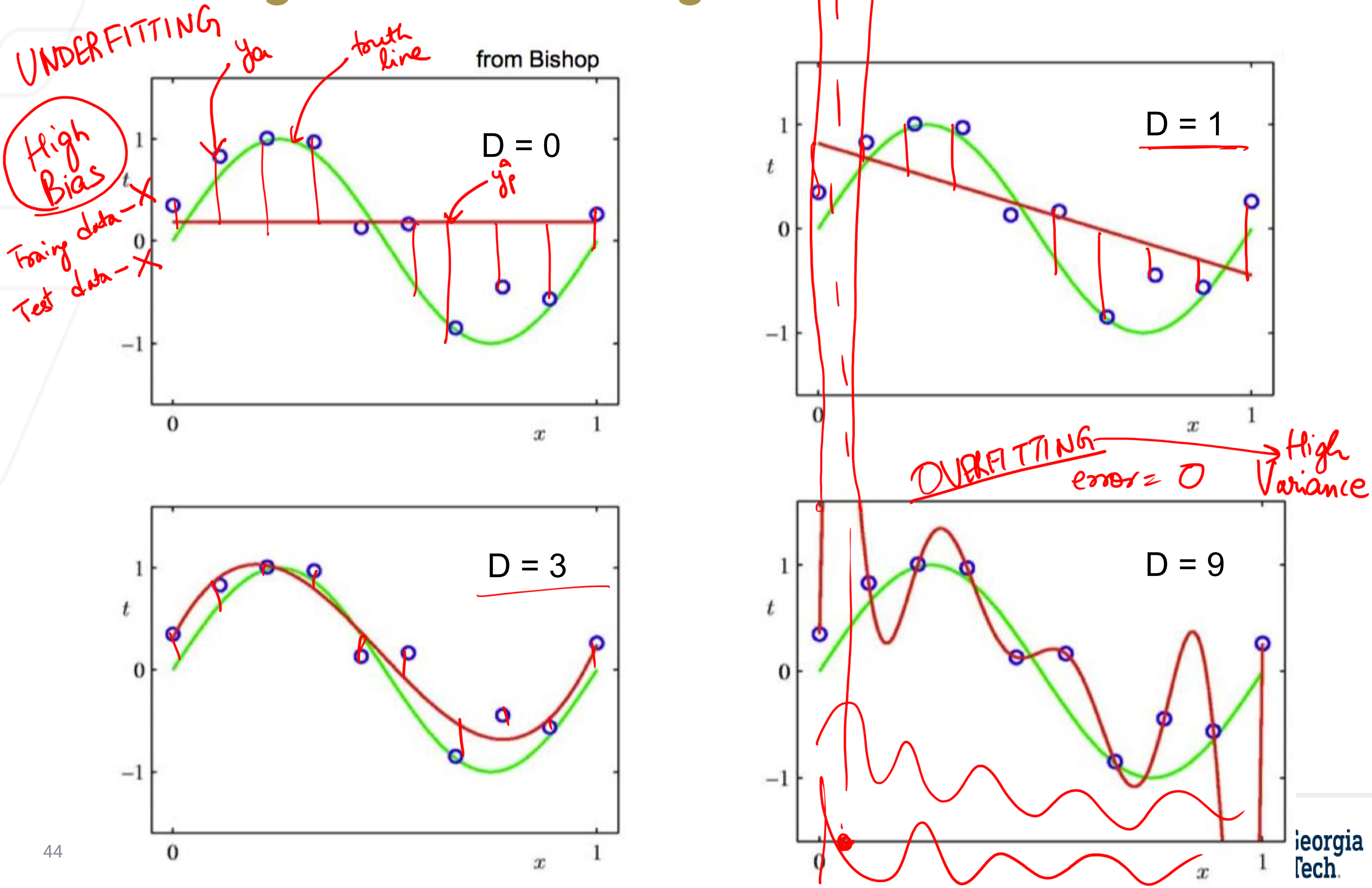
$\hat{y}_p$

We are fitting a D-dimensional hyperplane in a D+1 dimensional hyperspace (in above example a 2D plane in a 3D space). That hyperplane really is 'flat' / 'linear' in 3D. It can be seen a non-linear regression (a curvy line) in our 2D example in fact it is a flat surface in 3D.
So the fact that it is mentioned that the model is linear in parameters, it is shown here.





Increasing the Maximal Degree



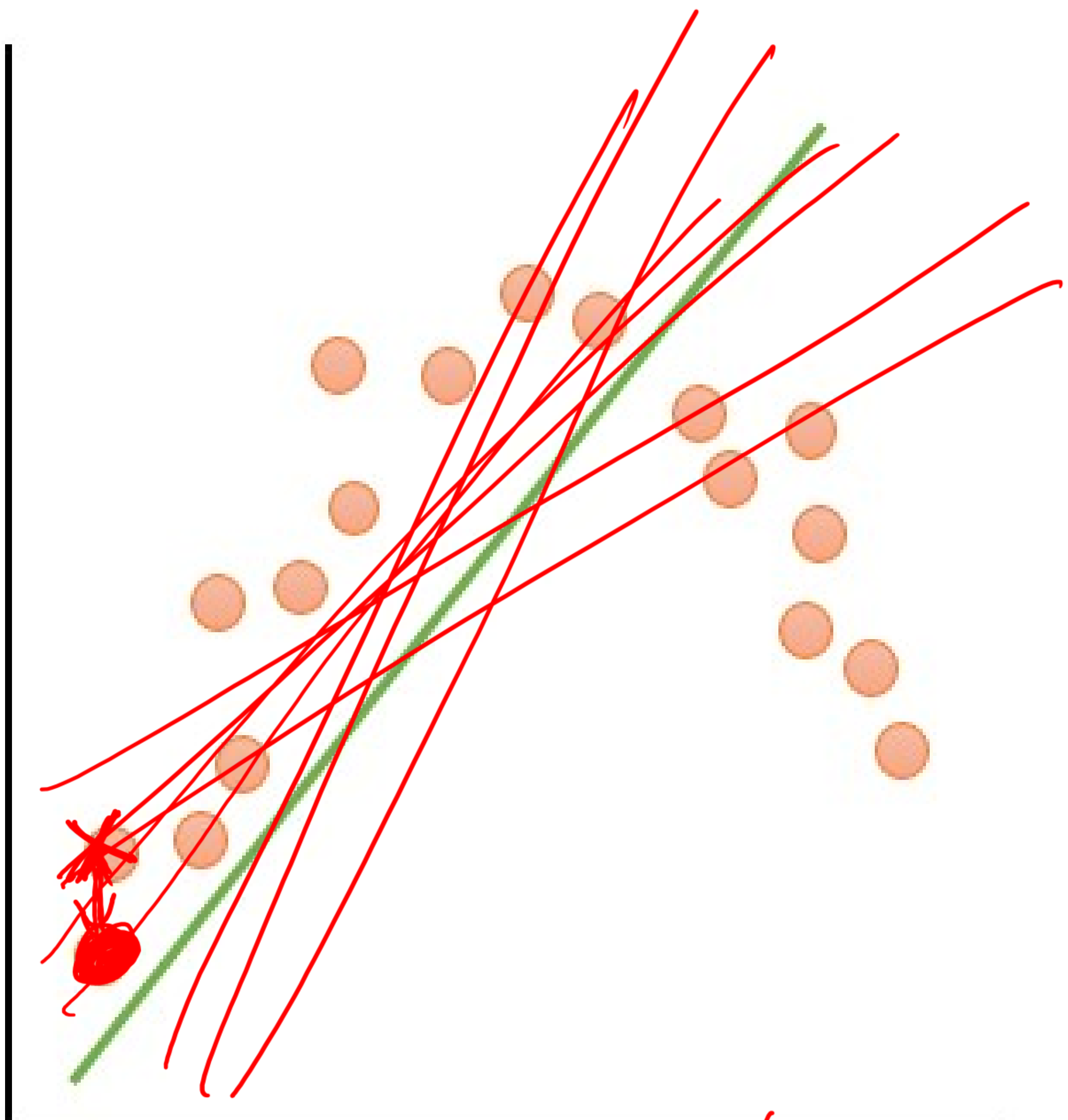
Georgia Tech

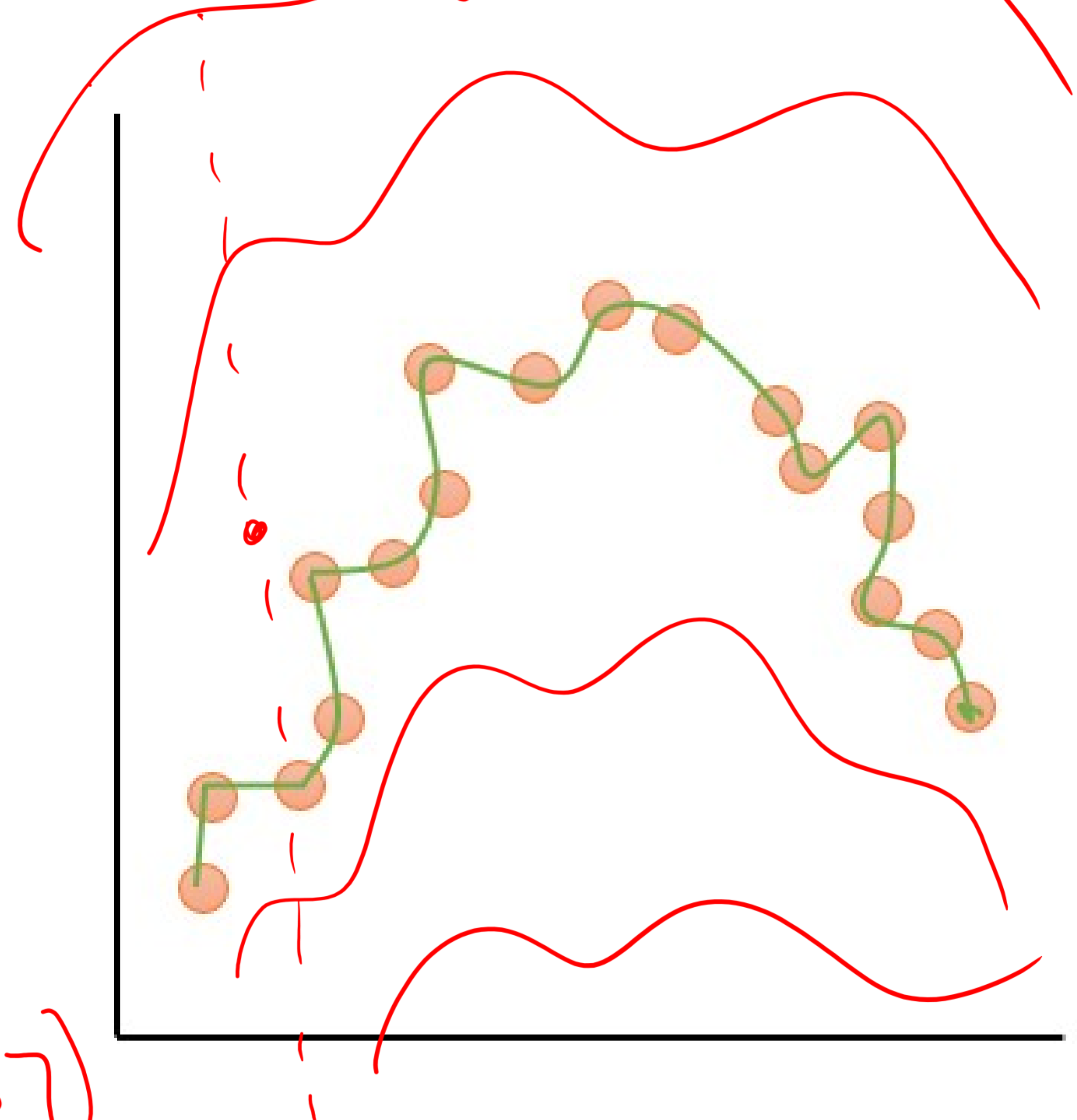
Subsample
↓
10 datasets ← $\hat{y}_p$

High Bias

$(\hat{y}_p - y_a)$

Low Bias





High Bias → $\left(y_a - E[\hat{y}_p]\right)$

high

What high variance means?

- **Variance** measures how much the model's predictions **change** if you train it on a different sample of same data.
- A **high-variance** model is *very sensitive* to small changes in the training set.
  → Even a slightly different dataset produces a very different fitted curve.
- Overfitting with high maximal degree leads to high variance

Bias-Variance Trade off

Animation

We will have multiple prediction values (i.e. through Cross validation) $E[y_p]$

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{N}\big(\underbrace{y^{\{i\}}}_{y_a} - \underbrace{x^{\{i\}}\theta}_{y_p}\big)^2 = E\left[(y_a - y_p)^2\right]$$

$$(y_a - y_p)^2 = \big(\underbrace{y_a - E[y_p]}_{a} + \underbrace{E[y_p] - y_p}_{b}\big)^2$$

scalar scalar scalar vector scalar

$$= \big(\overset{\text{scalar}}{y_a} - \overset{\text{scalar}}{E[y_p]}\big)^2 + \big(\overset{\text{scalar}}{E[y_p]} - \overset{\text{vector}}{y_p}\big)^2 + 2\big(y_a - E[y_p]\big)\cancel{\big(\overset{\text{scalar}}{E[y_p]} - y_p\big)}$$

$E[E[y_p]] = E[y_p]$

$E[y_p]$

$E\left[(E[y_p] - y_p)^2\right]$

$= \underbrace{(y_a - E[y_p])^2}_{b} +$

Why $E[2(y_a - E[y_p])(E[y_p] - y_p)] = 0$?

$y_a - E[y_p]$ is a scalar, therefore $E\left[y_a - E[y_p]\right] = y_a - E[y_p]$

$$E[2(y_a - E[y_p])(E[y_p] - y_p)]$$

$$= 2(y_a - E[y_p])E[E[y_p] - y_p]$$

$$= 2(y_a - E[y_p])\left(E\left[E[y_p]\right] - E[y_p]\right)$$

$$= 2(y_a - E[y_p])(E[y_p] - E[y_p]) = 0$$

Bias-Variance Trade off

Animation

We will have multiple prediction values (i.e. through Cross validation) $E[y_p]$

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{N}\left(y^{\{i\}} - x^{\{i\}}\theta\right)^2 = E\left[\left(\mathrm{y_a} - \mathrm{y_p}\right)^2\right]$$

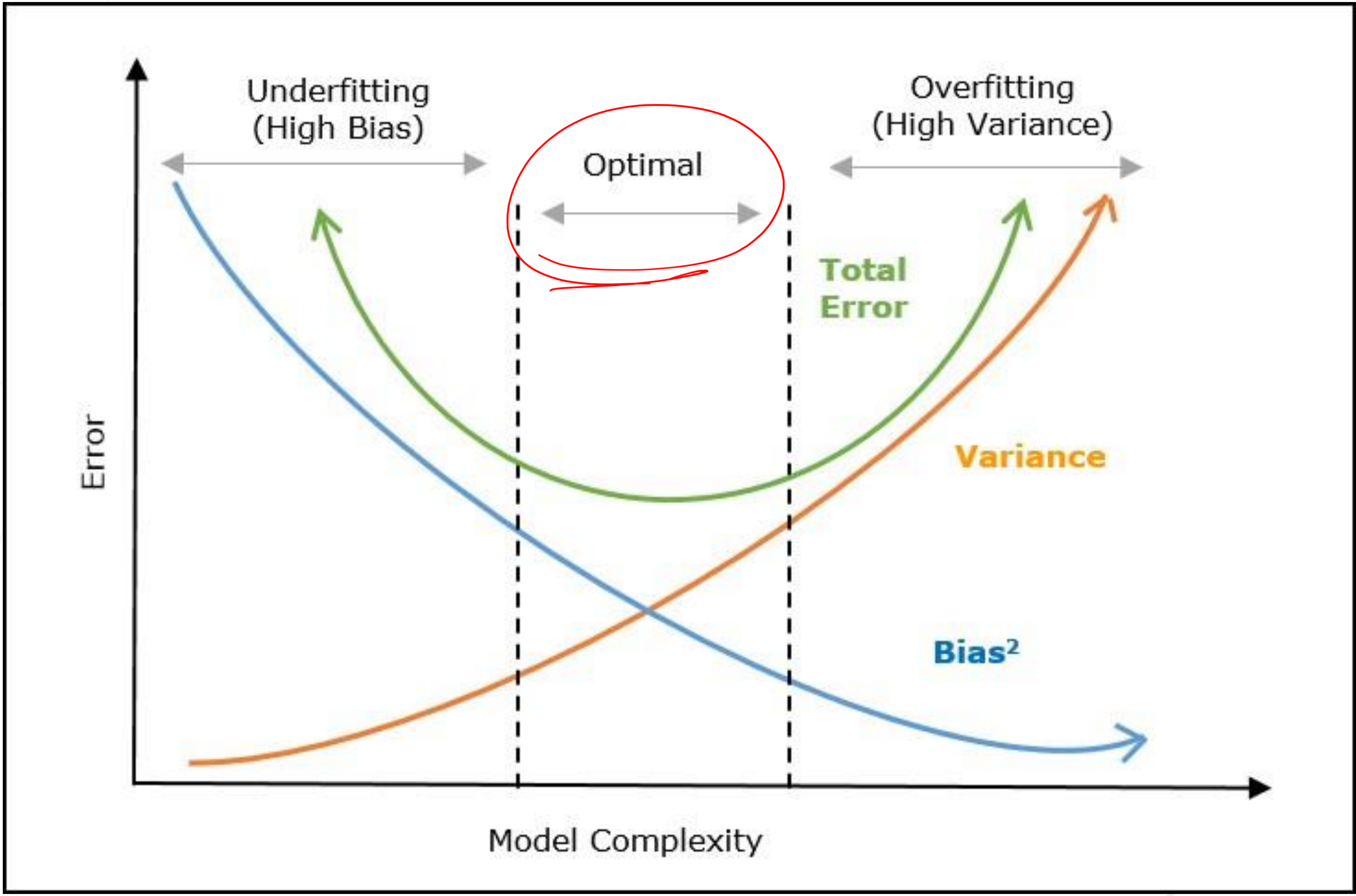
$$\left(\mathrm{y_a} - \mathrm{y_p}\right)^2 = \left(y_a - E[y_p] + E[y_p] - y_p\right)^2$$

$$= \left(y_a - E[y_p]\right)^2 + \left(E[y_p] - y_p\right)^2 + 2\left(y_a - E[y_p]\right)\left(E[y_p] - y_p\right)$$

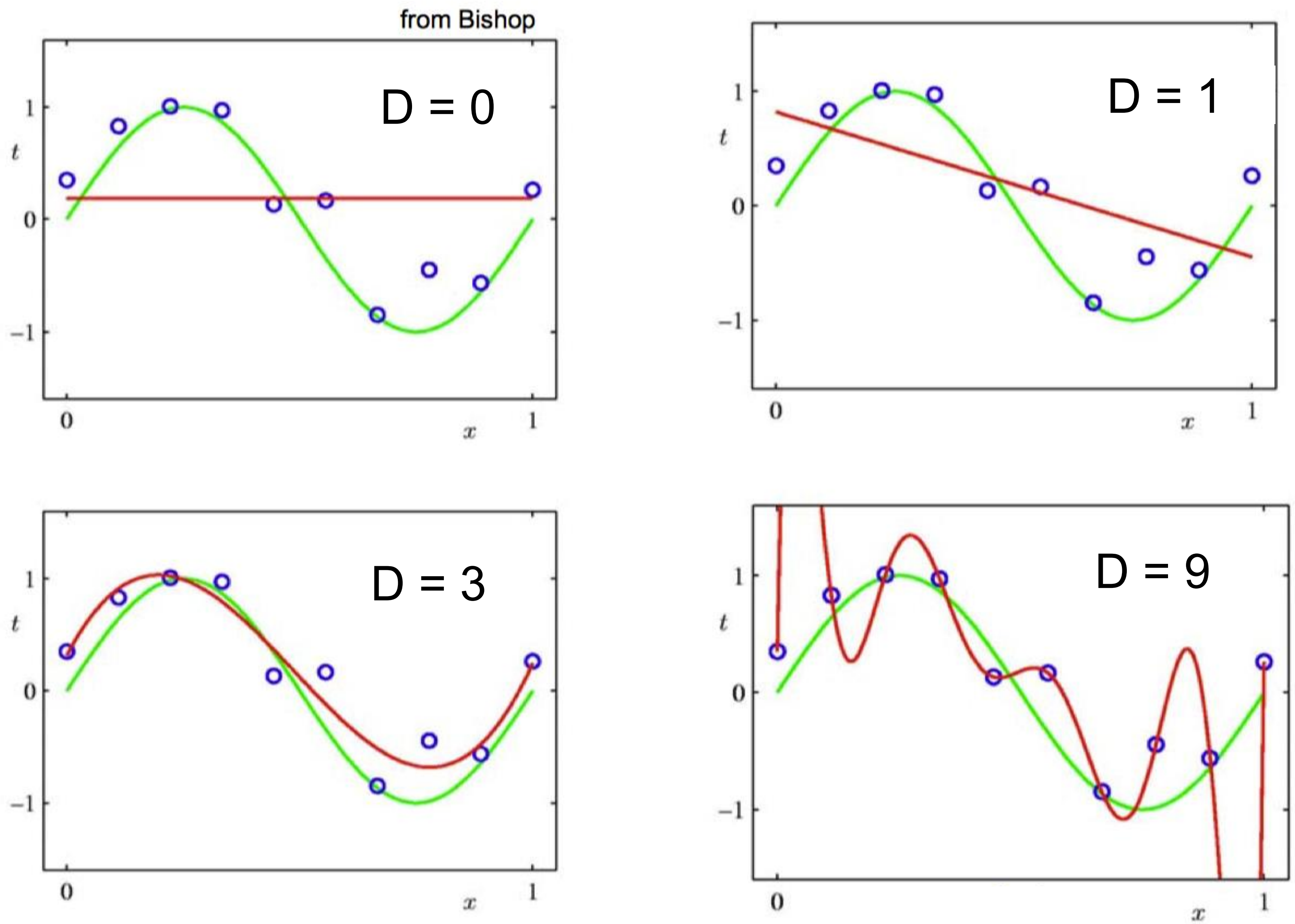
$$\mathbf{E}\left[\left(\boldsymbol{y_a} - \boldsymbol{y_p}\right)^{\mathbf{2}}\right] = \left(y_a - E[y_p]\right)^2 + E\left[\left(E[y_p] - y_p\right)^2\right]$$

$$= [Bias]^2 + Variance$$

$$= [true\ value - mean(predictions)]^2 - mean[(mean(prediction) - prediction)^2\ ]$$


Underfitting
(High Bias)
Optimal
Overfitting
(High Variance)
Total
Error
Variance
Bias²
Error
Model Complexity

Which One is Better?



- Can we increase the maximal polynomial degree to very large, such that the curve passes through all training points?

  - We will know the answer in next lecture.

Take-Home Messages

- Supervised learning paradigm
- Linear regression and least mean square
- Extension to high-order polynomials

x $\qquad$ x x^2 x^3 - - -

Quick Knowledge Check

1. Which of the following is added when extending linear regression to higher-order polynomial regression?
   A. More data points B. Higher-degree features C. More regularization D. Less variance
2. In polynomial regression, what happens when we increase the maximal degree d?
   A. The model becomes simpler B. The model becomes more flexible overfitting
   C. The model underfits D. The MSE always increases
3. A high-degree polynomial that fits all training points perfectly likely suffers from:
   A. Underfitting B. Overfitting C. Low variance D. Low flexibility
4. Why is overfitting considered a high-variance problem?
   A. The model changes significantly with new data B. It ignores noise
   C. It has high bias D. It's too simple to capture the trend
5. Which of the following best describes **bias** in the bias–variance tradeoff?
   A. Variability across datasets B. Sensitivity to noise C. Systematic error due to model simplicity D. Random error due to data splits

Quick Knowledge Check

1. What does high variance typically lead to?
   A. Stable predictions B. Poor generalization to new data C. Low flexibility D. Low training accuracy
2. In the bias–variance tradeoff, increasing model complexity generally:
   A. Increases bias and decreases variance B. Decreases bias and increases variance C. Decreases both bias and variance D. Increases both bias and variance
3. Which scenario indicates underfitting?
   A. High training error, high test error B. Low training error, high test error C. Low training error, low test error D. High training accuracy, low test accuracy